

LAPORAN TUGAS PROJECT
DESAIN BASIS DATA DAN OTOMASI SQL UNTUK SISTEM
MANAJEMEN RESTORAN



Disusun untuk memenuhi tugas terstruktur:

Mata kuliah : Basis Data

Dosen : Lutfiah Maharani Siniwi, S.Stat., M.Stat.

Kelompok F :

1. Galih Tri Prasetyo (K1D024028)
2. Dionisius Anthoni Pratama (K1D024031)
3. Rahma Aulia Fatma (K1D024032)
4. Aditya Febrio Setiawan (K1D024036)
5. Arroja Aliya Fadhilah (K1D024037)

KEMENTERIAN PENDIDIKAN TINGGI, SAINS DAN TEKNOLOGI
UNIVERSITAS JENDERAL SOEDIRMAN
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
S1 STATISTIKA
2026

KATA PENGANTAR

Puji syukur penulis panjatkan kepada Tuhan Yang Maha Esa atas rahmat dan karunia-Nya, sehingga laporan *project* yang berjudul “Desain Basis Data dan Otomasi SQL untuk Sistem Manajemen Restoran” ini dapat diselesaikan sesuai dengan yang diharapkan. Laporan ini membahas salah satu materi dari mata kuliah Basis Data, yaitu rekayasa basis data relasional terintegrasi yang merupakan materi penting untuk dapat dipahami mahasiswa/i dalam mata kuliah ini. Tidak hanya sebagai kompetensi dalam perkuliahan, akan tetapi pengetahuan akan normalisasi data dan otomasi objek sql ini juga penting sebagai bekal untuk dapat melakukan praktik dalam pengembangan sistem manajemen restoran serta membantu dalam merancang basis data yang efisien, konsisten, dan bebas dari redundansi data, sehingga dapat meningkatkan kualitas pengelolaan informasi di bidang sistem informasi.

Laporan ini dibuat dalam rangka memenuhi tugas mata kuliah Basis Data dalam pendalaman materi rekayasa basis data relasional terintegrasi. Tentunya penulis mendapatkan bimbingan, arahan, koreksi, serta saran dari dosen pembimbing. Untuk itu, rasa terima kasih yang teramat dalam penulis sampaikan kepada dosen mata kuliah Basis Data yang telah membimbing dan memberikan arahan selama proses penyusunan laporan ini. Dalam pembuatan laporan ini, tentunya masih jauh dari kata sempurna. Oleh karena itu, penulis harapkan kritik dan saran yang sifatnya membangun sebagai acuan penulis dalam pembuatan laporan-laporan *project* berikutnya. Demikian laporan ini dibuat, semoga dapat bermanfaat bagi penulis khususnya dan bagi pembaca umumnya.

Purwokerto, 26 Juni 2026

Penulis,

Kelompok F

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI	ii
DAFTAR TABEL	vi
DAFTAR GAMBAR	vii
BAB I PENDAHULUAN	1
A. Latar Belakang.....	1
B. Rumusan Masalah.....	1
C. Tujuan.....	2
BAB II ANALISIS KEBUTUHAN	3
A. Deskripsi Studi Kasus dan Ruang Lingkup Sistem	3
1. Deskripsi Studi Kasus	3
2. Ruang Lingkup Sistem.....	3
B. Identifikasi Pengguna	4
C. Daftar Kebutuhan Fungsional Terkait Data.....	5
D. <i>Business Rules</i>	6
BAB III PERANCANGAN	8
A. Entity Relationship Diagram (ERD).....	8
1. ERD Konseptual (<i>Crow's Foot</i>)	8
2. Pemetaan ERD ke Skema Relasional (<i>Logical Design</i>)	10
a. Pelanggan	10
b. Karyawan	11
c. Menu.....	11
d. Pesanan.....	11
e. Detail_Pesanan	11
f. Pembayaran	12
g. Meja.....	12
h. Reservasi	12
i. <i>Review</i>	12
j. Bahan_Baku	13
k. Menu Bahan	13
B. Penjelasan Entitas, Atribut, dan Kardinalitas	13

C.	Proses Normalisasi	16
1.	Bentuk Tidak Normal (UNF).....	16
2.	Bentuk Normal Pertama (1NF).....	17
3.	Bentuk Normal Kedua (2NF).....	18
4.	Bentuk Normal Ketiga (3NF)	18
D.	Data <i>Dictionary</i>	19
1.	Tabel Pelanggan.....	20
2.	Tabel Karyawan	20
3.	Tabel Meja	21
4.	Tabel Menu	21
5.	Tabel Bahan Baku.....	21
6.	Tabel Menu Bahan.....	21
7.	Tabel Pembayaran.....	22
8.	Tabel Reservasi	22
9.	Tabel Pesanan	23
10.	Tabel Detail Pesanan.....	23
11.	Tabel <i>Review</i>	24
BAB IV	IMPLEMENTASI.....	25
A.	Spesifikasi DBMS yang Digunakan	25
B.	<i>Script DDL (Data Definition Language)</i>	25
1.	Deskripsi Dataset	25
2.	<i>Create Table</i>	25
C.	<i>Script DML (Data Manipulation Language)</i>	26
1.	Deksripsi Dataset	26
2.	Data Pelanggan	26
3.	Data Karyawan.....	27
4.	Data Meja.....	27
5.	Data Menu.....	27
6.	Data Master Pembayaran	28
7.	Data Bahan Baku	28
8.	Data Menu Bahan	28
9.	Data Reservasi	29

10.	Data Pesanan	29
11.	Data Detail Pesanan	30
12.	Data <i>Review</i>	30
BAB V PENGUJIAN QUERY		32
A.	Query <i>Select</i>	32
1.	Query Sederhana	32
a.	Query Menampilkan Klasifikasi Harga Menu	32
b.	Query Penyaringan Kapasitas Meja	32
c.	Query Menampilkan Data Staf	32
2.	Query dengan JOIN	33
a.	Query Informasi Transaksi dan Pelayanan	33
b.	Query Rincian Item Penjualan Restoran	34
c.	Query Pemetaan Komposisi Produk	34
d.	Query Audit Ulasan dan Kepuasan Pelanggan	34
3.	Query dengan <i>Subquery</i> atau CTE	35
a.	Query Analisis Pelanggan Potensial (<i>Subquery</i>)	35
b.	Query Evaluasi Performa Produk Terbaik (CTE)	36
4.	Query dengan Agregasi, GROUP BY, dan HAVING	36
B.	View	37
1.	Deskripsi Logika	37
2.	<i>Script</i> dan Contoh Eksekusi	38
a.	Tabel Virtual Pendapatan Harian	38
b.	Tabel Virtual Cetak Struk	38
C.	Stored Procedure	39
1.	Deskripsi Logika	39
2.	Script dan Contoh Eksekusi	39
D.	Trigger	40
1.	Deskripsi Logika	40
2.	<i>Script</i> dan Contoh Eksekusi	40
E.	Function	41
1.	Deskripsi Fungsi	41
2.	<i>Script</i> dan Contoh Penggunaan	41

BAB VI PENUTUP.....	43
A. Kesimpulan.....	43
B. Saran	43
LAMPIRAN	45

DAFTAR TABEL

Tabel 2.1 Entitas Manajemen Restoran	3
Tabel 2.2 Identifikasi Pengguna, Peran, dan Hak Aksesnya	5
Tabel 2.3 Daftar Kebutuhan Fungsional	5
Tabel 2.4 Aturan Bisnis Sistem Manajemen Restoran	7
Tabel 3.1 Daftar Entitas dan Atribut.....	9
Tabel 3.2 Penjelasan Entitas dan Kardinalitas	13
Tabel 3.3 Bentuk Tidak Normal (UNF).....	17
Tabel 3.4 Bentuk Nomal Pertama (1NF)	17
Tabel 3.5 Tabel Pesanan	18
Tabel 3.6 Tabel Detail Pesanan	18
Tabel 3.7 Tabel Menu	19
Tabel 3.8 Tabel Pesanan	19
Tabel 3.9 Tabel Pelanggan.....	20
Tabel 3.10 Tabel Karyawan.....	20
Tabel 3.11 Tabel Meja	21
Tabel 3.12 Tabel Menu	21
Tabel 3.13 Tabel Bahan Baku.....	21
Tabel 3.14 Tabel Menu Bahan.....	22
Tabel 3.15 Tabel Pembayaran.....	22
Tabel 3.16 Tabel Reservasi.....	22
Tabel 3.17 Tabel Pesanan	23
Tabel 3.18 Tabel Detail Pesanan	23
Tabel 3.19 Tabel <i>Review</i>	24

DAFTAR GAMBAR

Gambar 3.1 ERD Notasi <i>Crow's Foot</i>	9
---	---

BAB I

PENDAHULUAN

A. Latar Belakang

Perkembangan teknologi informasi yang pesat telah mendorong berbagai sektor usaha, termasuk industri restoran, untuk beralih dari sistem pencatatan secara manual menuju sistem manajemen berbasis komputer. Sistem manajemen restoran yang efektif sangat bergantung pada pengelolaan data yang terstruktur, akurat, dan efisien. Data yang dikelola meliputi data menu, pelanggan, pesanan, bahan baku, transaksi pembayaran, hingga laporan penjualan harian. Tanpa pengelolaan data yang baik, restoran akan menghadapi berbagai masalah seperti redundansi data, inkonsistensi informasi, kesulitan dalam pembuatan laporan, serta lambatnya proses transaksi.

Salah satu pendekatan utama dalam merancang sistem manajemen data adalah dengan menggunakan basis data relasional. Basis data relasional menyimpan data dalam bentuk tabel-tabel yang saling terhubung melalui kunci utama (*primary key*) dan kunci asing (*foreign key*). Namun, perancangan basis data relasional yang buruk dapat menyebabkan anomali data, pemborosan ruang penyimpanan, dan kesulitan dalam pemeliharaan data. Untuk mengatasi hal tersebut, diperlukan normalisasi data, yaitu suatu proses untuk mengorganisasi struktur tabel agar memenuhi bentuk normal tertentu. Sehingga mengurangi redundansi dan meningkatkan integritas data.

B. Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, permasalahan utama dalam penelitian ini dirumuskan sebagai berikut:

1. Bagaimana bentuk rancangan basis data relasional untuk sistem manajemen restoran yang memenuhi prinsip normalisasi data hingga bentuk normal ketiga (3NF)?
2. Bagaimana penerapan objek sql berupa stored procedure, trigger, dan view untuk mengotomatisasi proses bisnis pada sistem manajemen restoran?
3. Bagaimana hasil integrasi antara normalisasi data dan otomasi objek sql dalam membangun basis data relasional yang terintegrasi, efisien, dan bebas dari redundansi serta anomali data?

C. Tujuan

Berdasarkan rumusan masalah di atas, maka tujuan dari penelitian ini adalah sebagai berikut:

1. Merancang basis data relasional untuk sistem manajemen restoran yang memenuhi prinsip normalisasi data hingga bentuk normal ketiga (3NF), sehingga menghasilkan struktur tabel yang efisien, tidak redundan, dan bebas dari anomali data.
2. Menerapkan objek sql berupa stored procedure, trigger, dan view untuk mengotomatisasi proses bisnis pada sistem manajemen restoran.
3. Mengintegrasikan konsep normalisasi data dengan otomasi objek sql dalam membangun basis data relasional yang terintegrasi, sehingga diperoleh sistem manajemen restoran yang efisien, akurat, dan mudah dipelihara.

BAB II

ANALISIS KEBUTUHAN

A. Deskripsi Studi Kasus dan Ruang Lingkup Sistem

1. Deskripsi Studi Kasus

Sistem yang dirancang dalam proyek ini adalah sistem manajemen restoran yang bertujuan untuk mengelola data operasional restoran secara terintegrasi. Berdasarkan data yang tersedia, restoran yang menjadi objek studi memiliki aktivitas meliputi pencatatan data pelanggan, data karyawan, pemesanan menu, pembayaran, reservasi meja, serta pengelolaan ulasan pelanggan. Setiap transaksi pemesanan mencatat menu yang dipesan beserta jumlah porsi, metode pembayaran yang digunakan, serta identitas pelanggan dan karyawan yang melayani. Selain itu, sistem juga mencatat hubungan antara menu dengan bahan baku penyusunnya untuk mendukung keperluan inventaris.

Data yang dijadikan acuan berasal dari dokumen berisi proses normalisasi (UNF, 1NF, 2NF, 3NF) yang mencakup 40 transaksi pesanan dengan berbagai macam menu, pelanggan berulang, dan beragam metode pembayaran. Hal ini menunjukkan bahwa sistem harus mampu menangani skenario transaksional yang cukup kompleks.

2. Ruang Lingkup Sistem

Pada sistem manajemen restoran, terdapat beberapa entitas yang terlibat dalam proses pengelolaan data. Setiap entitas memiliki peran dan cakupan informasi yang berbeda sesuai dengan kebutuhan sistem. Berikut merupakan daftar entitas beserta cakupan data yang dikelola dalam sistem.

Tabel 2.1 Entitas Manajemen Restoran

No	Entitas	Cakupan
1	Pelanggan	Menyimpan identitas pelanggan (id pelanggan, nama, email, nomor telepon, alamat).
2	Karyawan	Menyimpan data karyawan restoran (id karyawan, nama, jabatan, <i>shift</i> , nomor telepon).
3	Pesanan	Mencatat transaksi pemesanan (id pesanan, tanggal, total bayar, relasi ke pelanggan dan karyawan).
4	Detail Pesanan	Mencatat rincian menu dalam setiap pesanan makanan (id detail pesanan, jumlah porsi, catatan khusus, total bayar, relasi ke pesanan dan menu).

No	Entitas	Cakupan
5	Menu	Menyimpan daftar menu yang tersedia (id menu, nama menu, harga).
6	Bahan Baku	Menyimpan bahan baku yang digunakan dalam setiap menu. (id bahan, nama bahan, relasi ke menu).
7	Meja	Menyimpan data fisik meja (nomor meja, kapasitas maksimal).
8	Reservasi	Mencatat pemesanan meja oleh pelanggan (id reservasi, tanggal reservasi, jumlah tamu, relasi ke pelanggan dan meja).
9	Pembayaran	Menyimpan metode pembayaran (id bayar, metode pembayaran (tunai, qris, kartu, transfer), status bayar).
10	<i>Review</i>	Menampung ulasan pelanggan terhadap menu (id <i>Review</i> , rating, komentar, relasi ke pelanggan dan menu).

B. Identifikasi Pengguna

Berdasarkan analisis terhadap proses bisnis restoran dan skenario penggunaan sistem, diidentifikasi empat lapis pengguna yang akan berinteraksi dengan sistem. Meskipun tidak semua lapis pengguna mengakses basis data secara langsung, keberadaan mereka menentukan jenis data yang perlu dikelola dan hak akses yang diperlukan.

Tabel 2.2 Identifikasi Pengguna, Peran, dan Hak Aksesnya

Pengguna	Peran dan Hak Akses
Pelanggan	Melakukan pemesanan makanan, melakukan reservasi meja, dan memberikan ulasan (<i>Review</i>) terhadap menu yang telah dipesan. Pelanggan tidak dapat mengakses basis data secara langsung
Waiters	Bertanggung jawab untuk melayani pelanggan di area makan, mencatat pesanan makanan, menyampaikan catatan khusus pesanan, serta mengelola data reservasi meja. Hak akses waitres terhadap basis data meliputi operasi <i>create</i> dan <i>read</i> pada tabel Pesanan, Detail Pesanan, Reservasi, serta <i>read</i> pada tabel Menu dan Meja.
Kasir	Bertanggung jawab untuk memproses transaksi pembayaran, mencetak struk, serta memasukkan data pelanggan baru ke dalam sistem jika diperlukan. Hak akses kasir terhadap basis data meliputi operasi <i>create</i> , <i>read</i> , dan <i>update</i> pada tabel Pembayaran, <i>read</i> dan <i>update</i> pada tabel Pesanan, serta <i>read</i> pada tabel Pelanggan dan Menu.
Manajer Restoran	Memiliki hak akses penuh terhadap data master yang meliputi data karyawan, data menu, data bahan baku, dan data meja. Selain itu, manajer dapat melihat seluruh laporan operasional, seperti laporan penjualan makanan, laporan reservasi meja, laporan kinerja karyawan, serta rating dan ulasan pelanggan untuk keperluan evaluasi bisnis. Hak akses manajer bersifat <i>read-only</i> terhadap tabel transaksional kecuali untuk keperluan koreksi data tertentu.

C. Daftar Kebutuhan Fungsional Terkait Data

Pada sistem manajemen restoran, terdapat sejumlah kebutuhan fungsional yang harus dipenuhi agar sistem dapat mendukung proses operasional secara optimal. Kebutuhan tersebut menggambarkan fungsi yang harus dimiliki sistem dan disajikan pada tabel berikut.

Tabel 2.3 Daftar Kebutuhan Fungsional

Kode	Kebutuhan fungsional
KF-01	Sistem harus mampu melakukan operasi <i>create</i> , <i>read</i> , <i>update</i> , dan <i>delete</i> (CRUD) terhadap data pelanggan.
KF-02	Sistem harus mampu melakukan operasi CRUD terhadap data karyawan.
KF-03	Sistem harus mampu mencatat pesanan makanan yang mencakup id pesanan, tanggal pemesanan, total bayar, serta hubungan ke pelanggan dan karyawan yang melayani.

Kode	Kebutuhan fungsional
KF-04	Sistem harus mampu mencatat detail pesanan makanan yang mencakup jumlah porsi, catatan khusus (misalnya tingkat kepedasan atau pantangan alergi), total bayar per item, serta hubungan ke pesanan dan menu.
KF-05	Sistem harus mampu menyimpan dan mengelola data menu yang mencakup id menu, nama menu, dan harga.
KF-06	Sistem harus mampu menyimpan data bahan baku yang mencakup id bahan dan nama bahan, beserta hubungan <i>many-to-many</i> dengan menu (setiap menu dapat tersusun dari banyak bahan baku, dan setiap bahan baku dapat digunakan pada banyak menu).
KF-07	Sistem harus mampu menyimpan data meja yang mencakup id meja, nomor meja, kapasitas maksimal, dan status ketersediaan meja.
KF-08	Sistem harus mampu mencatat reservasi meja yang mencakup id reservasi, tanggal reservasi, jumlah tamu, serta hubungan ke pelanggan dan ke meja.
KF-09	Sistem harus mampu mencatat pembayaran yang mencakup id pembayaran, total bayar, metode pembayaran, serta hubungan ke pesanan dan karyawan pemroses.
KF-10	Sistem harus mampu menampung ulasan pelanggan (<i>Review</i>) yang mencakup id <i>Review</i> , rating (bilangan bulat 1-5), komentar, serta hubungan ke pelanggan dan menu.
KF-11	Sistem harus dapat menghitung total bayar suatu pesanan secara otomatis berdasarkan akumulasi hasil perkalian antara harga menu dan jumlah porsi yang tercantum dalam detail pesanan.
KF-12	Sistem harus dapat menghasilkan laporan daftar pesanan makanan dalam rentang waktu tertentu, laporan reservasi meja per periode, serta laporan rata-rata rating per menu.

D. *Business Rules*

Dalam rangka mengontrol alur kerja (*workflow*) dan menjaga konsistensi data di dalam sistem restoran, diperlukan spesifikasi aturan bisnis yang tegas. Aturan ini menjadi landasan dalam menentukan keterkaitan antar-tabel serta batasan nilai input yang diizinkan oleh sistem. Deskripsi lengkap mengenai aturan bisnis (BR-01 sampai BR-12) tersebut dijabarkan pada tabel sebagai berikut.

Tabel 2.4 Aturan Bisnis Sistem Manajemen Restoran

Kode	Aturan Bisnis
BR-01	Satu pesanan makanan dapat terdiri atas satu atau lebih detail pesanan. Setiap detail pesanan merepresentasikan satu menu dengan jumlah porsi tertentu.
BR-02	Jumlah porsi pada detail pesanan minimal bernilai 1 (satu). Nilai kurang dari 1 tidak diperbolehkan.
BR-03	Pesanan makanan tidak diwajibkan memiliki keterkaitan dengan reservasi meja. Pesanan dapat berdiri sendiri untuk melayani skenario <i>take away</i> atau <i>walk-in</i> tanpa menggunakan meja.
BR-04	Satu reservasi meja hanya berlaku untuk satu meja pada satu waktu (tanggal dan jam). Dalam periode waktu yang sama, satu meja tidak boleh dipeservasi oleh lebih dari satu pelanggan.
BR-05	Kapasitas meja yang tercantum dalam data meja harus lebih besar atau sama dengan jumlah tamu yang tercantum dalam reservasi.
BR-06	Rating pada tabel <i>Review</i> harus berupa bilangan bulat dalam rentang 1 hingga 5. Nilai di luar rentang tersebut tidak diperbolehkan.
BR-07	Ulasan (<i>Review</i>) hanya dapat diberikan untuk menu yang pernah dipesan oleh pelanggan yang bersangkutan. Hal ini dijamin melalui keberadaan pesanan yang menghubungkan pelanggan dan menu tersebut.
BR-08	Seorang pelanggan dapat memiliki banyak pesanan makanan, banyak reservasi meja, dan banyak ulasan (<i>Review</i>).
BR-09	Seorang karyawan dapat melayani banyak pesanan makanan dan banyak transaksi pembayaran.
BR-10	Satu menu dapat tersusun dari satu atau lebih bahan baku. Satu bahan baku dapat digunakan pada banyak menu.
BR-11	Metode pembayaran yang diakui oleh sistem hanya meliputi: tunai, qris, kartu debit/kredit, dan transfer bank. Nilai lain di luar keempat metode tersebut tidak diperbolehkan.
BR-12	Shift karyawan hanya boleh berisi salah satu dari tiga nilai berikut: pagi, siang, atau sore.

BAB III

PERANCANGAN

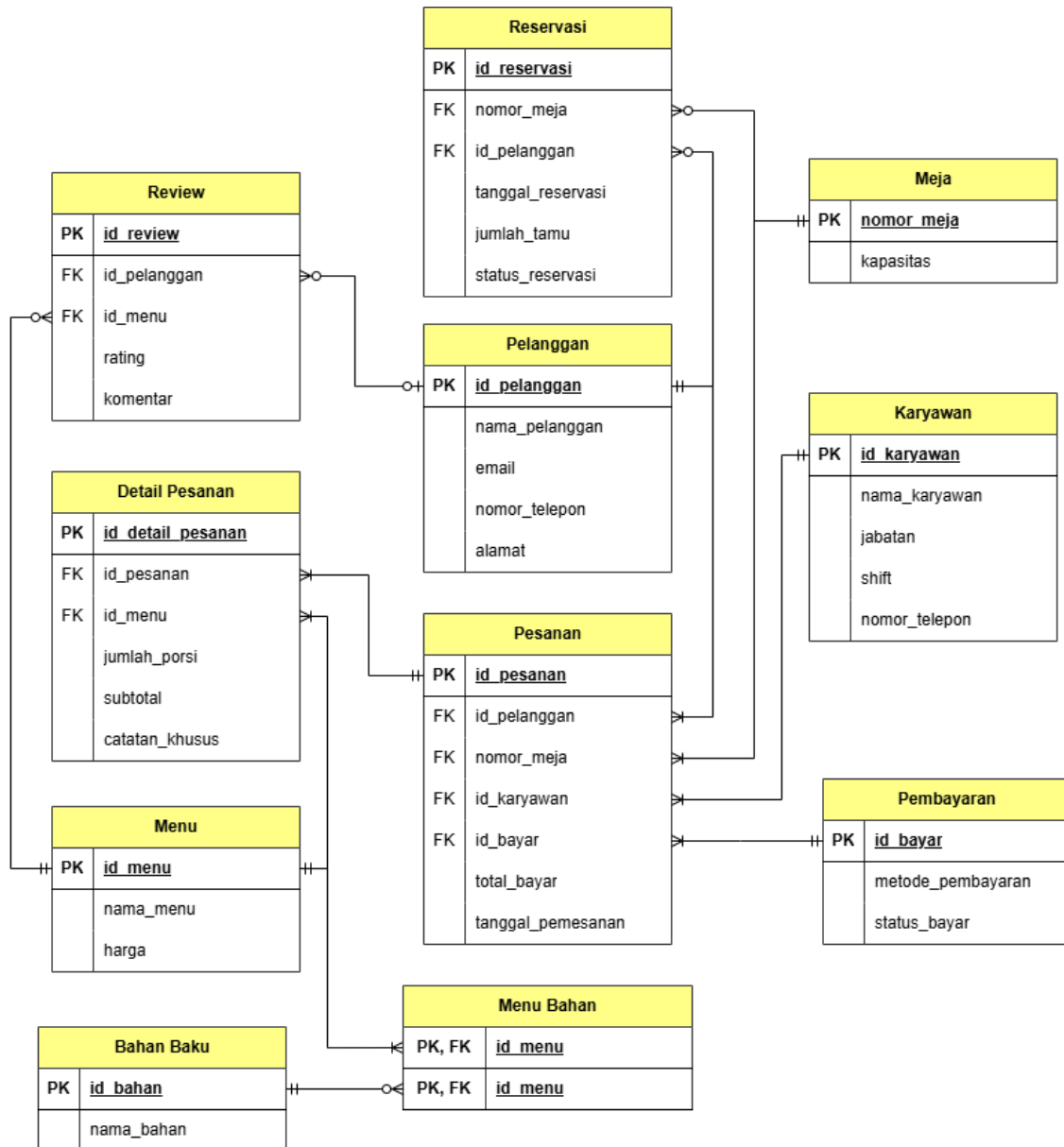
A. *Entity Relationship Diagram (ERD)*

Entity relationship diagram (ERD) merupakan model konseptual yang digunakan untuk menggambarkan struktur data dan hubungan antar entitas dalam sistem manajemen restoran. ERD berfungsi sebagai dasar dalam proses perancangan basis data relasional karena mampu memvisualisasikan kebutuhan data yang telah diidentifikasi pada tahap analisis kebutuhan. Melalui ERD, hubungan antar entitas dapat direpresentasikan secara jelas sehingga memudahkan proses implementasi ke dalam sistem manajemen basis data relasional.

Pada proyek ini, ERD dirancang menggunakan notasi *crow's foot* karena notasi tersebut mampu menunjukkan kardinalitas hubungan antar entitas secara lebih informatif. Sistem manajemen restoran yang dirancang mencakup pengelolaan pelanggan, karyawan, menu, pesanan, detail pesanan, pembayaran, reservasi meja, bahan baku, serta ulasan pelanggan. Keseluruhan entitas saling terintegrasi untuk mendukung proses bisnis restoran secara menyeluruh.

3. ERD Konseptual (*Crow's Foot*)

Berikut adalah ERD konseptual sistem manajemen restoran yang menampilkan sebelas entitas utama beserta seluruh atribut dan hubungan antar entitasnya menggunakan notasi *crow's foot*:



Gambar 3.1 ERD Notasi *Crow's Foot*

Berdasarkan rancangan konseptual pada Gambar 3.1, struktur basis data ini tersusun atas sebelas entitas (termasuk master data dan tabel penghubung) yang saling terintegrasi untuk mendukung seluruh proses bisnis restoran. Guna memberikan spesifikasi teknis yang lebih detail mengenai fungsi, *primary key* (PK), serta *foreign key* (FK) dari masing-masing komponen, berikut disajikan tabel daftar entitas dan atribut.

Tabel 3.1 Daftar Entitas dan Atribut

Entitas	Atribut
Pelanggan	id_pelanggan (PK), nama_pelanggan, email, nomor_telepon, alamat.

Entitas	Atribut
Menu	id_menu (PK), nama_menu, harga.
Pesanan	id_pesanan (PK), id_pelanggan (FK), nomor_meja (FK), id_karyawan (FK), id_bayar (FK), total_bayar, tanggal_pemesanan.
Detail Pesanan	id_detail_pesanan (PK), id_pesanan (FK), id_menu (FK), jumlah_porsi, subtotal, catatan_khusus.
Review	id_Review (PK), id_pelanggan (FK), id_menu (FK), rating, komentar.
Reservasi	id_reservasi (PK), id_pelanggan (FK), nomor_meja (FK), tanggal_reservasi, jumlah_tamu, status_reservasi.
Meja	nomor_meja (PK), kapasitas.
Karyawan	id_karyawan (PK), nama_karyawan, jabatan, nomor_telepon.
Pembayaran	id_bayar (PK), metode_pembayaran, status_bayar.
Bahan Baku	id_bahan (PK), nama_bahan.
Menu Bahan	id_menu (PK, FK), id_bahan (PK, FK).

Catatan: Entitas Menu Bahan merupakan tabel penghubung (*junction table*) yang merealisasikan relasi *many-to-many* antara entitas Menu dan Bahan Baku. *Primary Key* pada entitas ini bersifat komposit (*composite key*) yang terdiri dari id_menu dan id_bahan.

2. Pemetaan ERD ke Skema Relasional (*Logical Design*)

Setelah ERD konseptual selesai dirancang, langkah selanjutnya adalah melakukan pemetaan ke dalam skema relasional. Proses ini bertujuan untuk mengubah model konseptual menjadi struktur tabel yang dapat diimplementasikan pada sistem manajemen basis data. Skema relasional yang dihasilkan adalah sebagai berikut:

a. Pelanggan

Entitas Pelanggan berfungsi sebagai tabel utama untuk menyimpan seluruh data diri pelanggan restoran. Data ini digunakan sebagai referensi dalam proses pemesanan, reservasi meja, serta pemberian ulasan terhadap menu. Skema relasional untuk entitas Pelanggan adalah sebagai berikut:

```
Pelanggan(id_pelanggan, nama_pelanggan, email, nomor_telepon, alamat)
PK : id_pelanggan
```

b. Karyawan

Entitas Karyawan digunakan untuk menyimpan data seluruh pegawai restoran, termasuk nama, jabatan, shift kerja, dan nomor telepon. Data karyawan ini dihubungkan dengan proses pemesanan dan pembayaran untuk mengetahui siapa yang menangani transaksi. Skema relasional untuk entitas Karyawan adalah sebagai berikut:

Meja(nomor_meja, kapasitas)

PK : nomor_meja

c. Menu

Entitas Menu berfungsi sebagai tabel master yang menyimpan daftar seluruh makanan dan minuman yang tersedia di restoran beserta harga per porsi. Menu ini menjadi referensi utama dalam detail pesanan dan ulasan pelanggan. Skema relasional untuk entitas Menu adalah sebagai berikut:

Menu(id_menu, nama_menu, harga)

PK : id_menu

d. Pesanan

Entitas Pesanan merupakan tabel utama yang mencatat seluruh transaksi pemesanan yang dilakukan oleh pelanggan. Tabel ini menghubungkan pelanggan, karyawan, meja, dan pembayaran dalam satu kesatuan transaksi. Skema relasional untuk entitas Pesanan adalah sebagai berikut:

Pesanan(id_pesanan, id_pelanggan, nomor_meja, id_karyawan, id_bayar, total_bayar, tanggal_pemesanan)

PK : id_pesanan

FK : id_pelanggan

FK : nomor_meja

FK : id_karyawan

FK : id_bayar

e. Detail_Pesanan

Entitas Detail Pesanan digunakan untuk menyimpan rincian menu yang dipesan dalam setiap transaksi, termasuk jumlah porsi, subtotal, dan catatan khusus dari pelanggan. Tabel ini menjadi penghubung antara pesanan dan menu. Skema relasional untuk entitas Detail Pesanan adalah sebagai berikut:

Detail_Pesanan(id_detail_pesanan, id_pesanan, id_menu, jumlah_porsi, subtotal, catatan_khusus)

PK : id_detail_pesanan

FK : id_pesanan

FK : id_menu

f. Pembayaran

Entitas Pembayaran digunakan untuk menyimpan data metode dan status pembayaran dari setiap transaksi pesanan. Setiap pembayaran memiliki relasi one-to-one dengan pesanan yang bersangkutan. Skema relasional untuk entitas Pembayaran adalah sebagai berikut:

Pembayaran(id_bayar, metode_pembayaran, status_bayar)

PK : id_bayar

g. Meja

Entitas Meja digunakan untuk menyimpan data seluruh meja yang tersedia di restoran beserta kapasitas maksimal setiap meja. Data ini diperlukan dalam proses reservasi dan penempatan pelanggan saat melakukan pemesanan. Skema relasional untuk entitas Meja adalah sebagai berikut:

Meja(nomor_meja, kapasitas)

PK : nomor_meja

h. Reservasi

Entitas Reservasi digunakan untuk mencatat seluruh data pemesanan meja yang dilakukan oleh pelanggan sebelum datang ke restoran. Data ini mencakup tanggal reservasi, jumlah tamu, serta status reservasi. Skema relasional untuk entitas Reservasi adalah sebagai berikut:

Reservasi(id_reservasi, id_pelanggan, nomor_meja, tanggal_reservasi, jumlah_tamu, status_reservasi)

PK : id_reservasi

FK : id_pelanggan

FK : nomor_meja

i. Review

Entitas *Review* digunakan untuk menyimpan ulasan dan penilaian (rating) yang diberikan oleh pelanggan terhadap menu yang telah dipesan. Data ini bermanfaat untuk analisis kepuasan pelanggan dan evaluasi kualitas menu. Skema relasional untuk entitas *Review* adalah sebagai berikut:

Review(id_Review, id_pelanggan, id_menu, rating, komentar)

PK : id_Review

FK : id_pelanggan

FK : id_menu

j. Bahan_Baku

Entitas Bahan Baku digunakan untuk menyimpan data seluruh bahan baku yang digunakan dalam pembuatan menu restoran. Data ini menjadi dasar untuk mengelola komposisi bahan pada setiap menu serta mendukung perencanaan persediaan bahan. Skema relasional untuk entitas Bahan Baku adalah sebagai berikut:

```
Bahan_Baku(id_bahan, nama_bahan)
PK : id_bahan
```

k. Menu Bahan

Entitas Menu Bahan merupakan tabel penghubung (junction table) yang merealisasikan relasi many-to-many antara entitas Menu dan Bahan Baku. Selain berfungsi sebagai penghubung, tabel ini juga menyimpan informasi mengenai jumlah penggunaan bahan baku pada setiap menu. Skema relasional untuk entitas Menu Bahan adalah sebagai berikut:

```
Menu_Bahan(id_menu, id_bahan)
PK : (id_menu, id_bahan)
FK : id_menu
FK : id_bahan
```

B. Penjelasan Entitas, Atribut, dan Kardinalitas

Entitas merupakan objek atau konsep nyata dalam sistem yang memiliki data untuk disimpan. Setiap entitas memiliki atribut yang menggambarkan karakteristiknya, serta hubungan dengan entitas lain yang dinyatakan dalam kardinalitas. Berikut adalah penjelasan rinci mengenai entitas, atribut, dan kardinalitas pada Sistem Manajemen Restoran.

Tabel 3.2 Penjelasan Entitas dan Kardinalitas

Entitas	Primary Key	Deskripsi	Relasi Utama	Kardinalitas
Pelanggan	id_pelanggan	Menyimpan data pelanggan yang melakukan pemesanan.	Pelanggan Pesanan -	1 : M
Pelanggan	id_pelanggan	Menyimpan data pelanggan yang melakukan reservasi.	Pelanggan Reservasi -	1 : M

Entitas	Primary Key	Deskripsi	Relasi Utama	Kardinalitas
Pelanggan	id_pelanggan	Menyimpan data pelanggan yang melakukan pemberian ulasan.	Pelanggan - <i>Review</i>	1 : M
Karyawan	id_karyawan	Menyimpan data pegawai restoran yang melayani pelanggan.	Karyawan - Pesanan	1 : M
Karyawan	id_karyawan	Menyimpan data pegawai restoran yang memproses transaksi.	Karyawan - Pembayaran	1 : M
Pesanan	id_pesanan	Menyimpan data transaksi pemesanan makanan dan minuman.	Pesanan - Detail Pesanan	1 : M
Pesanan	id_pesanan	Menyimpan data transaksi pemesanan makanan dan minuman.	Pesanan - Pembayaran	1 : 1
Detail Pesanan	id_detail_pesanan	Menyimpan rincian menu yang dipesan dalam suatu transaksi.	Detail Pesanan - Menu	M : 1
Menu	id_menu	Menyimpan daftar makanan dan minuman yang tersedia di restoran.	Menu - Detail Pesanan	1 : M
Menu	id_menu	Menyimpan daftar makanan dan minuman yang tersedia di restoran.	Menu - <i>Review</i>	1 : M
Menu	id_menu	Menyimpan daftar makanan dan minuman yang tersedia di restoran.	Menu - Bahan Baku	M : N

Entitas	Primary Key	Deskripsi	Relasi Utama	Kardinalitas
Pembayaran	id_bayar	Menyimpan data pembayaran setiap transaksi pesanan.	Pembayaran - Pesanan	1 : 1
Reservasi	id_reservasi	Menyimpan data pemesanan meja oleh pelanggan.	Reservasi - Meja	M : 1
Meja	nomor_meja	Menyimpan data meja beserta kapasitas yang tersedia di restoran.	Meja - Reservasi	1 : M
Review	id_Review	Menyimpan penilaian dan komentar pelanggan terhadap menu yang pernah dipesan.	Review - Menu	M : 1
Bahan Baku	id_bahan	Menyimpan data bahan baku yang digunakan dalam pembuatan menu.	Bahan Baku - Menu (via Menu_Bahan)	M : N
Menu Bahan	id_menu, id_bahan	Tabel penghubung untuk relasi <i>many-to-many</i> antara Menu dan Bahan Baku, serta menyimpan jumlah penggunaan bahan.	Manu Bahan - Menu	M : 1
Menu Bahan	id_menu, id_bahan	Tabel penghubung untuk relasi <i>many-to-many</i> antara Menu dan Bahan Baku, serta menyimpan jumlah penggunaan bahan.	Manu Bahan - Menu	M : 1

Keterangan Kardinalitas:

1:M (*One-to-Many*) : Satu entitas di sisi kiri dapat berhubungan dengan banyak entitas di sisi kanan.

M:1 (*Many-to-One*) : Banyak entitas di sisi kiri berhubungan dengan satu entitas di sisi kanan.

1:1 (*One-to-One*) : Satu entitas di sisi kiri berhubungan dengan tepat satu entitas di sisi kanan.

M:N (*Many-to-Many*) : Banyak entitas di sisi kiri berhubungan dengan banyak entitas di sisi kanan, yang direalisasikan melalui tabel penghubung.

Berdasarkan Tabel 3.2 dan skema relasional yang telah dijabarkan, Sistem Manajemen Restoran terdiri atas sebelas entitas yang saling terintegrasi untuk mendukung proses operasional restoran. Sebagian besar hubungan antarentitas memiliki kardinalitas *one-to-many* (1:M), seperti hubungan antara pelanggan dengan pesanan, pelanggan dengan reservasi, serta pesanan dengan detail pesanan. Selain itu, terdapat hubungan *many-to-many* (M:N) antara entitas Menu dan Bahan_Baku yang direalisasikan melalui entitas penghubung Menu_Bahan. Struktur relasi tersebut memungkinkan penyimpanan data yang lebih terorganisasi, meminimalkan redundansi data, serta mendukung penerapan prinsip normalisasi hingga bentuk normal ketiga (3NF).

C. Proses Normalisasi

Normalisasi adalah proses pengorganisasian struktur tabel untuk mengurangi redundansi data dan menghindari anomali (kesalahan) saat dilakukan operasi penambahan, pengubahan, atau penghapusan data. Pada laporan ini menerapkan normalisasi hingga Bentuk Normal Ketiga (3NF) dengan tahapan sebagai berikut.

1. Bentuk Tidak Normal (UNF)

Sebelum dilakukan proses normalisasi, data berada dalam kondisi mentah yang dikenal sebagai Bentuk Tidak Normal. Pada tahap ini, struktur data masih sangat sederhana dan disusun berdasarkan catatan transaksi asli, namun belum memenuhi kaidah basis data relasional karena terdapat atribut yang menyimpan banyak nilai dalam satu kolom (*multi-value*) serta pengulangan kelompok data dalam satu baris *record*. Sebagai contoh, pada transaksi dengan kode PS01, kolom *id_menu* berisi nilai "M01, M02" dan kolom *nama_menu* berisi "Nasi Goreng, Es Teh" secara bersamaan, yang menunjukkan bahwa satu pesanan dapat mencakup beberapa menu sekaligus. Kondisi ini membuat

setiap sel tidak bersifat atomik, sehingga menyulitkan proses pencarian data, pembuatan filter laporan, serta berpotensi menimbulkan inkonsistensi informasi ketika terjadi pembaruan pada salah satu item pesanan. Oleh karena itu, data pada tahap UNF ini wajib direstrukturisasi sebelum dapat diproses lebih lanjut ke tahap normalisasi berikutnya. Berikut merupakan contoh tabel pada tahap Bentuk Tidak Normal (UNF).

Tabel 3.3 Bentuk Tidak Normal (UNF)

ID pesanan	Nama	Tanggal	Menu	Harga	Total
PS01	Andi	2026-01-10	Nasi Goreng, Es Teh	35000, 15000	50000
PS03	Citra	2026-01-11	Ayam Bakar, Es Buah	45000, 22000	67000

2. Bentuk Normal Pertama (1NF)

Transformasi dari UNF ke bentuk normal pertama dilakukan dengan memecah nilai-nilai ganda pada kolom-kolom tersebut ke dalam baris-baris yang terpisah, sehingga setiap sel hanya berisi satu nilai atomik (*atomic value*) dan setiap baris data bersifat unik. Pada implementasinya, transaksi PS01 yang semula hanya terdiri atas satu baris akan dipecah menjadi beberapa baris baru, misalnya baris pertama mewakili menu M01 (nasi goreng) dan baris kedua mewakili menu M02 (es teh), sehingga hubungan antara pesanan dan setiap item menu menjadi jelas. Meskipun secara struktural telah memenuhi syarat 1NF, tabel hasil transformasi ini masih menyimpan redundansi data yang cukup tinggi. Informasi seperti nama pelanggan, tanggal pemesanan, nomor meja, dan nama karyawan masih terulang untuk setiap item menu yang dipesan, yang menandakan bahwa data ini masih rentan terhadap anomali saat dilakukan operasi pengubahan atau penghapusan data di kemudian hari. Berikut merupakan tabel hasil transformasi ke Bentuk Normal Pertama (1NF).

Tabel 3.4 Bentuk Normal Pertama (1NF)

ID pesanan	Nama	Tanggal	Menu	Harga	Total
PS01	Andi	2026-01-10	Nasi Goreng	35000	50000
PS01	Andi	2026-01-10	Es Teh	15000	50000
PS03	Citra	2026-01-11	Ayam Bakar	45000	67000
PS03	Citra	2026-01-11	Es Buah	22000	67000

3. Bentuk Normal Kedua (2NF)

Memasuki tahap bentuk normal kedua, sebuah tabel harus terlebih dahulu memenuhi ketentuan 1NF dan yang lebih penting, tidak boleh memiliki ketergantungan parsial (*partial dependency*). Ketergantungan parsial terjadi ketika terdapat kolom non-kunci yang hanya bergantung pada sebagian dari *composite primary key* (kunci utama gabungan), bukan pada keseluruhan bagian kunci tersebut. Pada struktur 1NF, *composite primary key* secara logis terbentuk dari gabungan *id_pesanan*, *id_menu*, dan *id_bahan_baku*. Ditemukan bahwa kolom *tanggal_pemesanan* dan *total_bayar* hanya bergantung pada *id_pesanan*, sedangkan *nama_menu* dan *harga* hanya bergantung pada *id_menu*, sehingga kondisi ini secara jelas melanggar aturan 2NF. Untuk mengatasi permasalahan tersebut, dilakukan proses dekomposisi atau pemecahan tabel besar menjadi beberapa tabel entitas yang lebih spesifik, seperti tabel Pelanggan, Karyawan, Meja, Menu, Bahan_Baku, dan Pesanan. Dengan adanya pemisahan ini, setiap kolom non-kunci kini memiliki ketergantungan yang penuh terhadap *primary key* di masing-masing tabel barunya, sehingga struktur data menjadi lebih terorganisir dan mengurangi duplikasi informasi. Berikut merupakan contoh tabel-tabel hasil dekomposisi pada tahap Bentuk Normal Kedua (2NF).

Tabel 3.5 Tabel Pesanan

ID pesanan	Nama	Tanggal	Total
PS01	Andi	2026-01-10	50000
PS03	Citra	2026-01-11	67000

Tabel 3.6 Tabel Detail Pesanan

ID Detail Pesanan	ID Pesanan	Menu	Harga	Total
D01	PS01	Nasi Goreng	35000	50000
D02	PS01	Es Teh	15000	50000
D04	PS03	Ayam Bakar	45000	67000
D05	PS03	Es Buah	22000	67000

4. Bentuk Normal Ketiga (3NF)

Bentuk normal ketiga merupakan bentuk yang mensyaratkan tabel telah memenuhi 2NF dan sekaligus menghilangkan adanya ketergantungan transitif (*transitive*

dependency). Ketergantungan transitif adalah kondisi di mana sebuah kolom non-kunci bergantung pada kolom non-kunci lainnya, bukan secara langsung pada *primary key*, sehingga membentuk rantai ketergantungan yang tidak diinginkan. Contohnya, pada tabel Menu, kolom nama_bahan memiliki rantai ketergantungan melalui id_bahan yang terikat pada id_menu; begitu pula pada tabel Pesanan, kolom metode_bayar bergantung pada id_bayar yang berantai dengan id_pesanan, serta kolom rating dan komentar yang bergantung pada id_Review. Untuk memenuhi prinsip 3NF, kolom-kolom yang mengalami ketergantungan transitif ini dipindahkan ke tabel terpisah, seperti tabel Menu_Bahan (sebagai tabel jembatan untuk relasi *many-to-many* antara Menu dan Bahan_Baku), tabel Pembayaran, serta tabel Review. Hasil akhirnya, seluruh kolom non-kunci pada setiap tabel hanya bergantung secara langsung dan penuh terhadap *primary key* masing-masing, sehingga struktur basis data telah benar-benar efisien, terhindar dari anomali data, dan siap untuk diimplementasikan dalam sistem manajemen restoran. Berikut tabel-tabel final hasil pemisahan pada tahap bentuk normal ketiga (3NF).

Tabel 3.7 Tabel Menu

ID Menu	Nama	Harga
M01	Nasi Goreng	35000
M02	Es Teh	15000
M03	Ayam Bakar	45000
M13	Es Buah	22000

Tabel 3.8 Tabel Pesanan

ID Detail Pesanan	ID Pesanan	ID Menu	Total
D01	PS01	M01	35000
D02	PS01	M02	15000
D04	PS03	M03	45000
D05	PS03	M13	22000

D. Data Dictionary

Data *dictionary* (kamus data) merupakan dokumen pendukung yang berfungsi sebagai panduan komprehensif mengenai struktur basis data. Dokumen ini memuat

deskripsi terperinci dari setiap tabel, kolom, tipe data, ukuran, *constraint*, serta penjelasan fungsional masing-masing atribut. Keberadaan data *dictionary* sangat penting untuk menjamin keseragaman pemahaman antar pengembang, memudahkan proses pemeliharaan sistem, serta menjadi acuan dalam pengembangan fitur di masa mendatang. Berikut disajikan kamus data untuk seluruh tabel pada Sistem Manajemen Restoran yang telah dinormalisasi hingga 3NF.

1. Tabel Pelanggan

Tabel ini berfungsi untuk menyimpan data identitas dan informasi kontak seluruh pelanggan yang terdaftar dalam sistem, yang nantinya akan dirujuk oleh transaksi pemesanan maupun reservasi.

Tabel 3.9 Tabel Pelanggan

Kolom	Tipe Data	Kunci	Deskripsi
id_pelanggan	VARCHAR(10)	PK	Kode unik pelanggan, misal P001
nama_pelanggan	VARCHAR(100)	-	Nama lengkap pelanggan
email	VARCHAR(100)	-	Alamat email pelanggan
nomor_telepon	VARCHAR(15)	-	Nomor telepon pelanggan
alamat	TEXT	-	Alamat lengkap pelanggan

2. Tabel Karyawan

Tabel ini digunakan untuk mencatat data diri, jabatan, serta jadwal shift kerja setiap karyawan yang bertugas di restoran, guna mendukung pencatatan siapa yang melayani setiap transaksi.

Tabel 3.10 Tabel Karyawan

Kolom	Tipe Data	Kunci	Deskripsi
id_karyawan	VARCHAR(10)	PK	Kode unik karyawan, misal K01
nama_karyawan	VARCHAR(100)	-	Nama lengkap karyawan
jabatan	VARCHAR(50)	-	Jabatan (Waiter, Chef, Kasir, dll.)
shift	VARCHAR(20)	-	Shift kerja (Pagi, Siang, Sore)
nomor_telepon	VARCHAR(15)	-	Nomor telepon karyawan

3. Tabel Meja

Tabel ini merepresentasikan fasilitas fisik meja yang tersedia di restoran, mencakup kode nomor meja dan kapasitas maksimal tamu yang dapat ditampung pada setiap meja.

Tabel 3.11 Tabel Meja

Kolom	Tipe Data	Kunci	Deskripsi
nomor_meja	VARCHAR(10)	PK	Kode nomor meja, misal T01
kapasitas	INT	-	Jumlah maksimal orang per meja

4. Tabel Menu

Tabel ini berisi daftar seluruh item makanan dan minuman yang disediakan restoran, beserta harga jual per porsi yang menjadi acuan dalam perhitungan transaksi.

Tabel 3.12 Tabel Menu

Kolom	Tipe Data	Kunci	Deskripsi
id_menu	VARCHAR(10)	PK	Kode unik menu, misal M01
nama_menu	VARCHAR(100)	-	Nama menu makanan/minuman
harga	DECIMAL(10,2)	-	Harga per porsi dalam Rupiah

5. Tabel Bahan Baku

Tabel ini digunakan untuk mendata seluruh bahan baku yang diperlukan dalam proses produksi menu, sehingga manajemen dapat memantau komposisi dan kebutuhan persediaan.

Tabel 3.13 Tabel Bahan Baku

Kolom	Tipe Data	Kunci	Deskripsi
id_bahan	VARCHAR(10)	PK	Kode unik bahan baku, misal B01
nama_bahan	VARCHAR(100)	-	Nama bahan baku (Beras, Ayam, Teh)

6. Tabel Menu Bahan

Tabel ini berfungsi sebagai penghubung (relasi *many-to-many*) antara tabel Menu dan tabel Bahan Baku, untuk menentukan komposisi bahan apa saja yang digunakan dalam setiap hidangan.

Tabel 3.14 Tabel Menu Bahan

Kolom	Tipe Data	Kunci	Deskripsi
id_menu	VARCHAR(10)	PK, FK	Kode menu yang menggunakan bahan
id_bahan	VARCHAR(10)	PK, FK	Kode bahan yang digunakan dalam menu

7. Tabel Pembayaran

Tabel ini mencatat metode dan status transaksi pembayaran yang dilakukan oleh pelanggan, yang menjadi bagian penting dalam menyelesaikan siklus pemesanan.

Tabel 3.15 Tabel Pembayaran

Kolom	Tipe Data	Kunci	Deskripsi
id_bayar	VARCHAR(10)	PK	Kode unik pembayaran, misal PY01
metode_pembayaran	VARCHAR(30)	-	Metode: Tunai, QRIS, Kartu, Transfer
status_bayar	VARCHAR(20)	-	Status: Lunas, Pending, Batal

8. Tabel Reservasi

Tabel ini digunakan untuk menyimpan data pemesanan meja oleh pelanggan sebelum kedatangan, mencakup tanggal reservasi, jumlah tamu, serta status reservasi (misal: terkonfirmasi, selesai, atau batal).

Tabel 3.16 Tabel Reservasi

Kolom	Tipe Data	Kunci	Deskripsi
id_reservasi	VARCHAR(10)	PK	Kode unik reservasi, misal RS01
id_pelanggan	VARCHAR(10)	FK	Kode pelanggan pemesan
nomor_meja	VARCHAR(10)	FK	Kode meja yang dipesan
tanggal_reservasi	DATE	-	Tanggal reservasi
jumlah_tamu	INT	-	Jumlah orang yang hadir
status_reservasi	VARCHAR(20)	-	Status reservasi

9. Tabel Pesanan

Tabel ini merupakan *header* utama dari setiap transaksi pemesanan yang terjadi, mencatat pelanggan, karyawan pelayan, meja yang digunakan, serta total nilai keseluruhan pesanan.

Tabel 3.17 Tabel Pesanan

Kolom	Tipe Data	Kunci	Deskripsi
id_pesanan	VARCHAR(10)	PK	Kode unik pesanan, misal PS01
id_pelanggan	VARCHAR(10)	FK	Kode pelanggan pemesan
id_karyawan	VARCHAR(10)	FK	Kode karyawan pelayan
id_bayar	VARCHAR(10)	FK	Kode pembayaran yang digunakan
nomor_meja	VARCHAR(10)	FK	Kode meja yang digunakan
tanggal_pemesanan	DATE	-	Tanggal pemesanan
total_bayar	DECIMAL(10,2)	-	Total nilai pesanan (otomatis)

10. Tabel Detail Pesanan

Tabel ini menyimpan rincian *line item* dari setiap pesanan, meliputi menu apa saja yang dipesan, jumlah porsi, subtotal per *item*, serta catatan khusus dari pelanggan untuk setiap menu.

Tabel 3.18 Tabel Detail Pesanan

Kolom	Tipe Data	Kunci	Deskripsi
id_detail_pesanan	VARCHAR(10)	PK	Kode unik detail pesanan, misal D01
id_pesanan	VARCHAR(10)	FK	Kode pesanan induk
id_menu	VARCHAR(10)	FK	Kode menu yang dipesan
jumlah_porsi	INT	-	Jumlah porsi yang dipesan
subtotal	DECIMAL(10,2)	-	Harga per item ($\text{harga} \times \text{porsi}$)
catatan_khusus	TEXT	-	Catatan tambahan pesanan

11. Tabel *Review*

Tabel ini digunakan untuk menampung ulasan tertulis dan penilaian (*rating*) dari pelanggan terhadap pengalaman pemesanan mereka, yang berguna sebagai bahan evaluasi kualitas pelayanan dan produk restoran.

Tabel 3.19 Tabel *Review*

Kolom	Tipe Data	Kunci	Deskripsi
id_Review	VARCHAR(10)	PK	Kode unik <i>Review</i> , misal R001
id_pesanan	VARCHAR(10)	FK	Kode pesanan yang di <i>Review</i>
rating	INT	-	Skor 1–5 (1 = buruk, 5 = sangat baik)
komentar	TEXT	-	Ulasan tertulis pelanggan

BAB IV

IMPLEMENTASI

A. Spesifikasi DBMS yang Digunakan

Dalam implementasi sistem basis data manajemen restoran ini, kami memilih MySQL Workbench 8.0 CE sebagai *database management system* (DBMS) yang digunakan. MySQL Workbench 8.0 CE dipilih karena mysql workbench adalah alat administrasi database visual resmi yang dibuat Oracle Corporation khusus untuk mysql. Satu aplikasi untuk perancangan, pengembangan, dan administrasi database, semuanya berbasis grafis. Edisi *community*nya gratis dan *open source*, tanpa lisensi berbayar, sehingga cocok untuk proyek akademis maupun bisnis kecil. Workbench juga berjalan di Windows, Linux, dan macOS, jadi perbedaan sistem operasi antar anggota tim bukan masalah. Ditambah dokumentasi resminya lengkap dan komunitasnya besar, sehingga ketika ada kendala teknis, solusinya biasanya sudah tersedia di forum atau *stack overflow*.

B. Script DDL (*Data Definition Language*)

1. Deskripsi Dataset

Dataset yang digunakan dalam sistem manajemen restoran ini terdiri atas sebelas tabel relasional yang telah dinormalisasi hingga bentuk normal ketiga (3NF). Dataset ini mencakup data operasional restoran secara menyeluruh, meliputi data master (Pelanggan, Karyawan, Meja, Menu, Bahan_Baku, Pembayaran) dan data transaksional (Pesanan, Detail_Pesanan, Reservasi, Review, Menu_Bahan). Total dataset berisi 34 data pelanggan, 21 karyawan, 9 meja, 20 menu, 48 bahan baku, 39 pesanan, 41 detail pesanan, 16 reservasi, dan 20 ulasan. Seluruh tabel didefinisikan menggunakan perintah ddl dengan penerapan *primary key*, *foreign key*, serta *constraint* integritas data seperti NOT NULL, UNIQUE, dan CHECK untuk memastikan konsistensi data.

2. Create Table

Data definition language (DDL) digunakan untuk mendefinisikan struktur basis data, termasuk pembuatan tabel. CREATE TABLE adalah perintah sql yang digunakan untuk membuat sebuah tabel baru di dalam basis data. Perintah ini mendefinisikan nama tabel, nama setiap kolom beserta tipe datanya, serta aturan integritas data yang disebut *constraint*. *Constraint* yang umum digunakan meliputi PRIMARY KEY untuk menentukan kolom pengenal unik tiap baris, NOT NULL untuk mewajibkan kolom terisi,

UNIQUE untuk mencegah nilai duplikat, CHECK untuk membatasi nilai yang diizinkan, dan FOREIGN KEY untuk menghubungkan tabel satu dengan tabel lain sehingga integritas referensial antar tabel terjaga.

Berikut adalah contoh *script* ddl lengkap untuk implementasi sistem manajemen restoran yang telah dinormalisasi hingga 3NF:

Contoh :

```
CREATE TABLE Pelanggan (  
    id_pelanggan VARCHAR(10) PRIMARY KEY,  
    nama_pelanggan VARCHAR(100) NOT NULL,  
    email VARCHAR(100) UNIQUE,  
    no_telp VARCHAR(15),  
    alamat TEXT  
);
```

C. *Script DML (Data Manipulation Language)*

1. **Deksripsi Dataset**

Data manipulation language (DML) digunakan untuk memanipulasi data dalam tabel, meliputi operasi INSERT, UPDATE, DELETE, dan SELECT. Dataset yang digunakan dalam implementasi dml ini merupakan data simulasi operasional restoran. Dataset ini terdiri atas 34 *record* pelanggan yang tersebar dalam berbagai kota di Indonesia, 21 karyawan dengan jabatan beragam (manajer, waiter, chef, kasir, pencuci piring, dan OB) pada tiga *shift* kerja (pagi, siang, dan sore), 9 meja dengan kapasitas 2 hingga 6 orang, 20 item menu makanan dan minuman dengan rentang harga Rp10.000 hingga Rp85.000, serta 48 bahan baku. Data transaksional mencakup 39 pesanan utama dengan 41 item detail pesanan, 16 reservasi meja, 4 metode pembayaran (tunai, qris, kartu, transfer), dan 20 ulasan pelanggan. Seluruh nilai subtotal dan total bayar diisi secara otomatis melalui mekanisme trigger dan stored procedure, sehingga data dimasukkan dengan nilai 0 terlebih dahulu sebelum diperbarui oleh objek sql tersebut.

2. **Data Pelanggan**

Data pelanggan terdiri atas 34 *record* yang merepresentasikan pelanggan dari berbagai kota di Indonesia. Setiap *record* memuat id_pelanggan (format P001-P034), nama pelanggan, alamat email unik, nomor telepon, dan alamat domisili. Data ini menjadi fondasi referensial bagi seluruh transaksi pesanan, reservasi, dan ulasan dalam sistem.

```
INSERT INTO Pelanggan (id_pelanggan, nama_pelanggan, email, no_telp,
alamat) VALUES
('P001', 'andi', 'andi@email.com', '8267354890', 'Jakarta'),
('P002', 'budi', 'budi@email.com', '8987765356', 'Bandung'),
('P003', 'citra', 'citra@email.com', '8113456789', 'Surabaya'),
...
('P034', 'hendra', 'hendra@email.com', '08144567890', 'Banjarnegara');
```

3. Data Karyawan

Data karyawan mencakup 21 pegawai restoran yang terbagi dalam lima jabatan. Karyawan dijadwalkan dalam tiga *shift* kerja, yaitu pagi, siang, dan sore, kecuali Manajer yang tidak terikat *shift*. Setiap *record* memuat id_karyawan (format K01-K21), nama, jabatan, shift, dan nomor telepon. Data karyawan dirujuk oleh tabel Pesanan untuk mencatat siapa yang melayani setiap transaksi.

```
INSERT INTO Karyawan (id_karyawan, nama_karyawan, jabatan, shift,
nomor_telepon) VALUES
('K01', 'martin', 'Manajer', '-', '8924687243'),
('K02', 'adi', 'Waiter', 'Pagi', '8222333444'),
('K03', 'mina', 'Waiter', 'Sore', '8223456789'),
...
('K21', 'renata', 'Chef', 'Sore', '8337886520');
```

4. Data Meja

Data pelanggan terdiri atas 34 *record* yang merepresentasikan pelanggan dari berbagai kota di Indonesia. Setiap *record* memuat id_pelanggan (format P001-P034), nama lengkap, alamat email unik, nomor telepon, dan alamat domisili. Data ini menjadi fondasi referensial bagi seluruh transaksi pesanan, reservasi, dan ulasan dalam sistem.

```
INSERT INTO Meja (nomor_meja, kapasitas) VALUES
('T01', 6), ('T02', 2), ('T03', 2), ('T04', 4), ('T05', 4),
('T06', 2), ('T07', 2), ('T08', 4), ('T09', 2);
```

5. Data Menu

Data menu memuat 20 item makanan dan minuman yang tersedia di restoran, dengan rentang harga dari Rp10.000 hingga Rp85.000. Menu terdiri dari berbagai kategori, mulai dari makanan berat hingga minuman serta dessert. Harga setiap menu menjadi dasar perhitungan subtotal pada tabel Detail_Pesanan secara otomatis melalui trigger.

```
INSERT INTO Menu (id_menu, nama_menu, harga) VALUES
('M01', 'Nasi Goreng', 35000),
('M02', 'Es Teh', 15000),
```

```
('M03', 'Ayam Bakar', 45000),
...
('M20', 'Es Pisang Ijo', 17000);
```

6. Data Master Pembayaran

Data master pembayaran hanya terdiri atas 4 record yang merepresentasikan metode pembayaran yang diakui sistem sesuai *business rules* BR-11, yaitu tunai (PY01), qris (PY02), kartu (PY03), dan transfer bank (PY04). Seluruh *record* memiliki status_bayar bernilai lunas karena tabel ini berfungsi sebagai referensi metode, bukan pencatatan nominal transaksi. Setiap pesanan wajib merujuk ke salah satu dari keempat *record* ini melalui *foreign key* id_bayar pada tabel Pesanan.

```
INSERT INTO Pembayaran (id_bayar, metode_pembayaran, status_bayar)
VALUES
('PY01', 'Tunai', 'Lunas'),
('PY02', 'QRIS', 'Lunas'),
('PY03', 'Kartu', 'Lunas'),
('PY04', 'Transfer', 'Lunas');
```

7. Data Bahan Baku

Data bahan baku terdiri atas 48 item bahan yang digunakan dalam proses produksi berbagai menu restoran. Bahan baku mencakup kategori protein (ayam, gurame, udang, kepiting, cumi, bebek), karbohidrat (beras, mie, tepung), sayuran (sawi, kol, bawang, daun bawang), bumbu (cabai, garam, gula, kecap, madu), serta bahan minuman dan *dessert* (alpukat, mangga, sirsak, teh, kelapa, pisang, santan, keju). Data ini terhubung ke tabel Menu melalui tabel jembatan Menu_Bahan untuk mendukung pencatatan komposisi setiap menu.

```
INSERT INTO Bahan_Baku (id_bahan, nama_bahan) VALUES
('B01', 'Beras'), ('B02', 'Kecap'), ('B03', 'Bawang'), ('B04', 'Telur'),
('B05', 'Teh'), ('B06', 'Gula'), ('B07', 'Garam'), ('B08', 'Cabai'),
...
('B45', 'Bebek'), ('B46', 'Tempe'), ('B47', 'Pisang'), ('B48',
'Santan');
```

8. Data Menu Bahan

Data Menu_Bahan merupakan tabel jembatan yang merealisasikan relasi *many-to-many* antara tabel Menu dan tabel Bahan_Baku. Terdapat total 60 pasangan kombinasi id_menu dan id_bahan. Tabel ini memungkinkan sistem melacak komposisi bahan baku dari setiap menu yang tersedia untuk keperluan manajemen inventaris dapur.

```

INSERT INTO Menu_Bahan (id_menu, id_bahan) VALUES
('M01', 'B01'), ('M01', 'B04'), ('M01', 'B03'), ('M01', 'B02'), ('M01',
'B10'),
('M02', 'B05'), ('M02', 'B34'),
('M03', 'B19'), ('M03', 'B08'), ('M03', 'B03'), ('M03', 'B02'), ('M03',
'B01'),
('M04', 'B11'), ('M04', 'B04'), ('M04', 'B03'), ('M04', 'B02'), ('M04',
'B10'),
...
('M20', 'B47'), ('M20', 'B36'), ('M20', 'B34'), ('M20', 'B48');

```

9. Data Reservasi

Data reservasi mencatat 16 pemesanan meja yang dilakukan oleh pelanggan. Setiap *record* memuat id_reservasi (format RS01-RS16), id pelanggan pemesan, nomor meja yang dipesan, tanggal reservasi, jumlah tamu, dan status reservasi. Status reservasi memiliki nilai *default* 'pending' saat dibuat, dan akan diubah menjadi 'selesai' setelah pelanggan selesai dilayani. Beberapa pelanggan tercatat melakukan reservasi lebih dari satu kali, seperti pelanggan P001 (andi) dan P011 (kurnia) yang masing-masing memiliki dua data reservasi.

```

INSERT INTO Reservasi (id_reservasi, id_pelanggan, nomor_meja,
tanggal_reservasi, jumlah_tamu, status_reservasi) VALUES
('RS01', 'P001', 'T05', '2026-01-10', 2, 'Selesai'),
('RS02', 'P003', 'T01', '2026-01-11', 5, 'Selesai'),
('RS03', 'P005', 'T02', '2026-01-12', 2, 'Selesai'),
...
('RS17', 'P007', 'T05', '2026-01-22', 2, 'Selesai');

```

10. Data Pesanan

Data pesanan mencakup 39 *record* transaksi pemesanan. Setiap *record* memuat id_pesanan (format PS01-PS39), tanggal pemesanan, id pelanggan, id karyawan pelayan, metode pembayaran, nomor meja, serta total bayar yang diinisialisasi dengan nilai 0. Nilai total_bayar akan diperbarui secara otomatis oleh stored procedure HitungTotalPesanan setelah data detail pesanan dimasukkan. Sebagian pesanan terhubung dengan data reservasi meja, sementara beberapa lainnya merupakan pesanan *walk-in* tanpa reservasi (id_reservasi bernilai NULL).

```

INSERT INTO Pesanan (id_pesanan, id_pelanggan, id_reservasi,
id_karyawan, id_bayar, nomor_meja, tanggal_pemesanan, total_bayar)
VALUES

```

```
( 'PS01', 'P001', 'RS01', 'K02', 'PY01', 'T05', '2026-01-10', 0.00),
( 'PS02', 'P002', NULL, 'K12', 'PY02', 'T02', '2026-01-10', 0.00),
( 'PS03', 'P003', 'RS02', 'K13', 'PY03', 'T01', '2026-01-11', 0.00),
...
( 'PS40', 'P007', 'RS17', 'K13', 'PY01', 'T05', '2026-01-22', 0.00);
```

11. Data Detail Pesanan

Data detail pesanan memuat 41 *record* yang merupakan rincian item menu dari setiap transaksi pesanan. Setiap *record* mencatat `id_detail_pesanan` (format D01-D63), `id_pesanan` induk, `id_menu` yang dipesan, jumlah porsi, subtotal, serta catatan khusus dari pelanggan. Kolom subtotal diinisialisasi dengan nilai 0 dan akan dihitung otomatis oleh trigger `trg_auto_hitung_subtotal` saat data dimasukkan, berdasarkan perkalian harga menu dan jumlah porsi. Catatan khusus yang tercatat meliputi permintaan seperti pedas, kurang manis, bagian dada, dan biasa, mencerminkan preferensi pelanggan secara individual.

```
INSERT INTO Detail_Pesanan (id_detail_pesanan, id_pesanan, id_menu,
jumlah_porsi, subtotal, catatan_khusus) VALUES
( 'D01', 'PS01', 'M01', 2, 0, 'Pedas'),
( 'D02', 'PS01', 'M02', 1, 0, 'Kurang manis'),
( 'D03', 'PS02', 'M03', 1, 0, 'Bagian dada'),
...
( 'D63', 'PS40', 'M01', 1, 0, 'Biasa');
```

12. Data Review

Data ulasan (*Review*) memuat 22 *record* yang merupakan representasi umpan balik dari pelanggan terhadap menu makanan, fasilitas, maupun pelayanan restoran atas transaksi pesanan yang telah selesai diproses. Setiap *record* mencatat `id_Review` (format R001-R022), `id_pesanan` induk sebagai acuan keabsahan transaksi, nilai rating dalam skala numerik (1 hingga 5), serta komentar tertulis dari pelanggan. Komponen rating mencerminkan tingkat kepuasan konsumen secara objektif, sementara kolom komentar menampung masukan kualitatif yang bervariasi mulai dari pujian terhadap cita rasa masakan dan keramahan staf, hingga kritik membangun mengenai kebersihan meja dan tingkat kematangan hidangan guna menjadi bahan evaluasi manajemen dalam menjaga kualitas layanan restoran.

```
INSERT INTO Review (id_Review, id_pesanan, rating, komentar) VALUES
( 'R001', 'PS01', 5, 'Enak sekali'),
( 'R002', 'PS02', 4, 'Pelayanan bagus'),
```

```
('R003', 'PS04', 3, 'Agak lama'),  
.  
.  
.  
('R022', 'PS40', 4, 'Tempatnya nyaman');
```

BAB V

PENGUJIAN QUERY

A. Query Select

1. Query Sederhana

Untuk memastikan basis data dapat menyajikan informasi yg dibutuhkan oleh operasional restoran, dilakukan implementasi dan pengujian terhadap beberapa skenario query, mulai dari pencarian data sederhana hingga analisis data kompleks. Query sederhana ini digunakan untuk menarik atau menampilkan data dari satu tabel spesifik tanpa adanya modifikasi atau penggabungan kompleks.

a. Query Menampilkan Klasifikasi Harga Menu

Query ini digunakan untuk menarik data master menu sekaligus mengategorikan kelas harga setiap menu secara otomatis menggunakan FUNCTION bernama KlasifikasiHargaMenu. Berikut adalah *script* yang kami jalankan.

```
SELECT id_menu, nama_menu, harga, KlasifikasiHargaMenu(harga) AS  
tier_harga  
FROM Menu;
```

b. Query Penyaringan Kapasitas Meja

Query ini berfungsi untuk memfilter dan menampilkan data spesifik dari suatu tabel berdasarkan kriteria pembatas tertentu. Berikut adalah *script* yang kami jalankan.

```
SELECT nomor_meja, kapasitas FROM Meja  
WHERE kapasitas >= 3;
```

Query ini memanggil data kolom nomor_meja dan kapasitas dari tabel Meja. Klausula WHERE kapasitas >= 3 bertindak sebagai penyaring operasional, sehingga sistem hanya akan meloloskan dan menampilkan data meja yang memiliki daya tampung minimal 3 orang.

c. Query Menampilkan Data Staf

Query ini merupakan operasi proyeksi dasar (*basic projection*) yang digunakan untuk mengekstraksi kolom-kolom spesifik dari tabel tanpa memproses pembatasan baris data (*filter row*). Berikut adalah *script* yang kami jalankan.

```
SELECT nama_karyawan, jabatan FROM Karyawan;
```

Perintah sql di atas mengeksekusi proses pembacaan data langsung pada tabel Karyawan. Klausula SELECT secara spesifik memproyeksikan dua atribut utama, yaitu nama_karyawan dan jabatan, untuk ditampilkan ke dalam halaman hasil. Karena query

ini tidak dibatasi oleh klausa penyaring seperti WHERE, database akan menampilkan seluruh baris data staf yang terdaftar di dalam sistem secara utuh. *Output* dari query ini menyajikan data yang bersih dari informasi sensitif (seperti nomor telepon atau alamat tinggal karyawan), sehingga sangat ideal untuk digunakan sebagai penyajian laporan ringkas mengenai pemetaan komposisi dan distribusi posisi kerja staf di dalam organisasi restoran.

2. Query dengan JOIN

Operasi penggabungan multitabel (JOIN) digunakan untuk menyatukan kembali data yang telah terpisah akibat proses normalisasi basis data, guna menghasilkan laporan informasi yang utuh dan bernilai bagi manajemen restoran. Secara teknis, operasi ini bekerja dengan cara mencocokkan nilai atribut *foreign key* (FK) pada tabel transaksi dengan *primary key* (PK) pada tabel master terkait (seperti INNER JOIN atau LEFT JOIN). Melalui mekanisme eliminasi redundansi data ini, sistem dapat menyajikan informasi transaksional yang kaya akan konteks operasional, seperti menyandingkan kode nota pesanan langsung dengan nama pelanggan atau nama menu makanan yang bersangkutan.

a. Query Informasi Transaksi dan Pelayanan

Manajemen restoran membutuhkan laporan riwayat pesanan harian yang informatif untuk mengetahui pelanggan siapa saja yang bertransaksi beserta staf karyawan (kasir/pelayan) yang melayani transaksi tersebut. Berikut adalah *script* yang kami jalankan.

```
SELECT  pes.id_pesanan,    pes.tanggal_pemesanan,    p.nama_pelanggan,
        k.nama_karyawan
FROM    Pesanan pes
JOIN    Pelanggan p ON pes.id_pelanggan = p.id_pelanggan
JOIN    Karyawan k ON pes.id_karyawan = k.id_karyawan;
```

Query ini menggabungkan tabel Pesanan dengan dua tabel sekaligus, yaitu Pelanggan dan Karyawan. Proses relasi dilakukan secara simultan melalui pencocokan *foreign key* id_pelanggan dan id_karyawan yang terdapat pada tabel Pesanan. Hasil query berhasil mentransformasikan kode-kode id numerik yang kaku menjadi laporan operasional utuh yang menampilkan nama riil pelanggan dan karyawan terkait.

b. Query Rincian Item Penjualan Restoran

Bagian keuangan dan operasional dapur memerlukan laporan detail mengenai item menu apa saja yang terjual pada tiap transaksi, lengkap dengan informasi jumlah porsi serta subtotal biayanya untuk keperluan audit pendapatan. Berikut adalah *script* yang kami jalankan.

```
SELECT dp.id_detail_pesanan, pes.tanggal_pemesanan, m.nama_menu,
dp.jumlah_porsi, dp.subtotal
FROM Detail_Pesanan dp
JOIN Pesanan pes ON dp.id_pesanan = pes.id_pesanan
JOIN Menu m ON dp.id_menu = m.id_menu;
```

Query ini berfokus pada tabel objek penghubung Detail_Pesanan. Sistem melakukan multi-join dengan menarik data waktu transaksi dari tabel Pesanan via relasi id_pesanan, sekaligus menarik informasi nama produk dari tabel Menu melalui relasi id_menu. Eksekusi query ini menghasilkan laporan penjualan yang sangat mendalam per item menu yang dipesan pelanggan.

c. Query Pemetaan Komposisi Produk

Manajemen dapur dan bagian gudang persediaan membutuhkan laporan transparansi resep untuk melihat daftar bahan baku apa saja yang menyusun setiap menu makanan atau minuman yang dijual. Berikut adalah *script* yang kami jalankan.

```
SELECT m.nama_menu, bb.nama_bahan
FROM Menu m
JOIN Menu_Bahan mb ON m.id_menu = mb.id_menu
JOIN Bahan_Baku bb ON mb.id_bahan = bb.id_bahan;
```

Query ini digunakan untuk menjembatani hubungan *many-to-many* antara tabel Menu dan Bahan_Baku. Dengan memanfaatkan tabel *bridge* (penghubung) bernama Menu_Bahan, sistem mencocokkan id_menu dan id_bahan. *Output* query ini sangat penting untuk operasional *inventory control* guna memantau ketergantungan bahan baku terhadap menu yang ditawarkan.

d. Query Audit Ulasan dan Kepuasan Pelanggan

Tim manajemen kualitas (*quality control*) membutuhkan dokumen valid untuk meninjau rating dan ulasan pelanggan, guna memastikan ulasan tersebut benar-benar berdasarkan transaksi nyata yang valid. Berikut adalah *script* yang kami jalankan.

```
SELECT r.id_Review, p.nama_pelanggan, pes.id_pesanan, r.rating,
r.komentar
```

```
FROM Review r
JOIN Pesanan pes ON r.id_pesanan = pes.id_pesanan
JOIN Pelanggan p ON pes.id_pelanggan = p.id_pelanggan;
```

Query ini menghubungkan data dari tabel *Review* menuju tabel *Pesanan* berdasarkan kecocokan *id_pesanan*. Alur penggabungan kemudian diteruskan dari tabel *Pesanan* ke tabel *Pelanggan* melalui *id_pelanggan*. Melalui interkoneksi tiga arah ini, manajemen bisa mendapatkan data akuntabel mengenai nama konsumen dan nomor nota mana yang memberikan umpan balik beserta ulasan performanya.

3. Query dengan *Subquery* atau CTE

Operasi *subquery* dan *common table expression* (CTE) digunakan untuk mengisolasi logika perhitungan atau penyaringan data yang kompleks menjadi sebuah tabel virtual sementara sebelum diproses oleh query utama. Dalam arsitektur database restoran ini, teknik tersebut sangat krusial untuk menangani analisis data bertingkat seperti menghitung nilai rata-rata (*agregasi*) terlebih dahulu, lalu menyaring menu yang berada di atas ambang batas tersebut pada tahap berikutnya. Penggunaan CTE lewat klausa *WITH* lebih diutamakan dibandingkan *subquery* konvensional karena mampu memisahkan blok kode rumit ke bagian paling atas, sehingga meningkatkan keterbacaan (*readability*) struktur kode serta mempermudah proses pemeliharaan (*maintenance*) query laporan manajemen.

a. Query Analisis Pelanggan Potensial (*Subquery*)

Tim pemasaran (*marketing*) membutuhkan daftar data kontak pelanggan yang memiliki catatan transaksi bernilai besar (di atas Rp50.000) dalam satu kali kunjungan. Informasi ini akan digunakan sebagai basis data target promosi eksklusif atau program loyalitas restoran. Berikut adalah *script* yang kami jalankan.

```
SELECT p.nama_pelanggan, p.no_telp, trx_besar.total_bayar
FROM Pelanggan p
JOIN (
    SELECT id_pelanggan, total_bayar
    FROM Pesanan
    WHERE total_bayar > 50000
) AS trx_besar ON p.id_pelanggan = trx_besar.id_pelanggan;
```

Query ini menerapkan teknik *subquery* pada klausal *FROM* yang bertindak sebagai *derived table* (tabel turunan sementara) dengan alias *trx_besar*. Database pertama-tama akan mengisolasi dan menyaring data pada tabel *Pesanan* untuk mencari

id_pelanggan yang nilai total_bayar-nya melebihi nominal Rp50.000. Hasil penyaringan terisolasi tersebut kemudian diJOIN dengan tabel Pelanggan melalui kecocokan id_pelanggan. Pendekatan ini membuat proses pencarian menjadi lebih efisien karena operasi join hanya dilakukan pada baris data yang sudah lolos kualifikasi filter awal.

b. Query Evaluasi Performa Produk Terbaik (CTE)

Manajemen restoran ingin melakukan audit menu guna mempertahankan kualitas rasa dan pelayanan. Dapur membutuhkan laporan mengenai menu makanan atau minuman apa saja yang berhasil mendapatkan respons positif dari konsumen, dengan indikator nilai rata-rata rating ulasan di atas angka 3.0. Berikut adalah *script* yang kami jalankan.

```
WITH RataRataRating AS (  
    SELECT dp.id_menu, AVG(r.rating) AS avg_rating  
    FROM Review r  
    JOIN Pesanan pes ON r.id_pesanan = pes.id_pesanan  
    JOIN Detail_Pesanan dp ON pes.id_pesanan = dp.id_pesanan  
    GROUP BY dp.id_menu  
)  
SELECT m.nama_menu, r.avg_rating  
FROM Menu m  
JOIN RataRataRating r ON m.id_menu = r.id_menu  
WHERE r.avg_rating > 3.0;
```

Implementasi query ini memanfaatkan fungsi kustom CTE (*common table expression*) melalui klausa WITH untuk membangun struktur tabel virtual bernama RataRataRating. Di dalam blok CTE tersebut, sistem melakukan kalkulasi nilai rata-rata rating menggunakan fungsi agregasi AVG(r.rating) yang dikelompokkan berdasarkan id_menu lewat operasi GROUP BY setelah menghubungkan tabel Review, Pesanan, dan Detail_Pesanan. Setelah tabel bayangan CTE terbentuk di memori, query utama (*outer query*) langsung memanggil tabel tersebut untuk diJOIN dengan tabel Menu guna menyaring (WHERE r.avg_rating > 3.0) dan menampilkan visualisasi nama menu beserta nilai rating terbaiknya.

4. Query dengan Agregasi, GROUP BY, dan HAVING

Operasi agregasi yang dikombinasikan dengan klausa GROUP BY dan HAVING digunakan untuk melakukan analisis statistik data operasional dengan cara

mengelompokkan baris data yang memiliki karakteristik serupa ke dalam satu ringkasan informasi. Secara teknis, fungsi agregasi (seperti SUM, AVG, atau COUNT) bertugas melakukan kalkulasi matematis terhadap kelompok data yang telah didefinisikan oleh GROUP BY. Sementara itu, klausa HAVING bertindak sebagai filter tingkat lanjut untuk menyaring hasil kalkulasi agregat tersebut berdasarkan kriteria tertentu. Berbeda dengan klausa WHERE yang menyaring data di tingkat baris sebelum dikelompokkan, fungsi HAVING secara khusus bekerja setelah data selesai dikelompokkan, sehingga sangat ideal untuk menyusun laporan analisis metrik performa operasional yang membutuhkan penyaringan berdasarkan hasil perhitungan kumulatif data tertentu. Berikut adalah *script* yang kami jalankan.

```
SELECT      m.id_menu,      m.nama_menu,      SUM(dp.jumlah_porsi)      AS
total_porsi_terjual
FROM Detail_Pesanan dp
JOIN Menu m ON dp.id_menu = m.id_menu
GROUP BY m.id_menu, m.nama_menu
HAVING SUM(dp.jumlah_porsi) > 3;
```

Query ini bekerja dengan menghubungkan tabel Detail_Pesanan dan tabel Menu menggunakan relasi id_menu. Setelah data terhubung, fungsi GROUP BY m.id_menu, m.nama_menu akan mengelompokkan seluruh baris transaksi berdasarkan masing-masing item menu. Sistem kemudian menerapkan fungsi agregasi SUM(dp.jumlah_porsi) untuk menghitung total akumulasi porsi yang terjual untuk tiap kelompok menu tersebut. Terakhir, klausa HAVING SUM(dp.jumlah_porsi) > 3 bertindak sebagai penyaring pasca-agregasi untuk mengeleminasi menu yang sepi peminat dan hanya menampilkan menu yang total penjualannya berhasil menembus ambang batas di atas 3 porsi.

B. View

1. Deskripsi Logika

Operasi pembuatan tabel virtual (VIEW) digunakan untuk menyimpan query kompleks yang sering digunakan ke dalam satu objek basis data tiruan yang dapat dipanggil layaknya tabel biasa. Dalam arsitektur basis data restoran ini, VIEW bertindak sebagai lapisan abstraksi data yang tidak menyimpan data secara fisik di dalam memori, melainkan selalu mengeksekusi query relasional di bawahnya secara dinamis setiap kali dipanggil. Implementasi ini sangat krusial bagi manajemen dari sisi keamanan dan efisiensi operasional; sistem dapat menyembunyikan kolom sensitif (seperti data pribadi

karyawan atau detail struktur database asli) dan hanya menyajikan kumpulan data siap pakai yang sudah terfilter, seperti ringkasan pendapatan harian atau manifes meja yang terisi, sehingga mempermudah pengembang aplikasi dalam melakukan pemanggilan data tanpa perlu menulis ulang query panjang.

2. *Script* dan Contoh Eksekusi

a. **Tabel Virtual Pendapatan Harian**

Eksekutif dan pemilik restoran membutuhkan akses cepat dan berkala untuk memantau performa finansial bisnis tanpa harus menjalankan query agregasi yang rumit setiap harinya. Objek VIEW ini dibuat untuk menyajikan laporan ringkas mengenai total omset pendapatan dan volume transaksi yang terjadi pada tiap tanggal kalender. Berikut adalah *script* yang kami jalankan

```
CREATE VIEW View_Pendapatan_Harian AS
SELECT
    tanggal_pemesanan AS tanggal_transaksi,
    COUNT(DISTINCT id_pesanan) AS jumlah_transaksi,
    SUM(total_bayar) AS total_pendapatan
FROM Pesanan
GROUP BY tanggal_pemesanan;
```

Perintah di atas membangun sebuah tabel virtual permanen di dalam skema database dengan nama View_Pendapatan_Harian. Di dalam objek ini, sistem mengelompokkan data berdasarkan tanggal_pemesanan. Fungsi COUNT(DISTINCT id_pesanan) digunakan untuk menghitung total nota unik yang tercipta, sedangkan SUM(total_bayar) mengakumulasikan seluruh uang yang masuk pada hari tersebut. Saat pengguna memanggil perintah SELECT * FROM View_Pendapatan_Harian,, database secara dinamis menyajikan rangkuman laporan berkala yang bersih dan siap dianalisis.

b. **Tabel Virtual Cetak Struk**

Komponen kasir pada aplikasi restoran membutuhkan sumber data yang terintegrasi dan siap pakai untuk mencetak struk belanja fisik maupun digital secara real-time. VIEW ini dirancang untuk menyatukan seluruh komponen data transaksi yang terpisah agar pelayan dapat melihat rincian belanja konsumen dalam satu struktur data yang ringkas. Berikut adalah script yang kami jalankan.

```
CREATE VIEW View_Struk_Pesanan AS
SELECT
```

```

        pes.id_pesanan, p.nama_pelanggan, m.nama_menu, dp.jumlah_porsi,
        m.harga AS harga_satuan, dp.subtotal AS subtotal
FROM Pesanan pes
JOIN Pelanggan p ON pes.id_pelanggan = p.id_pelanggan
JOIN Detail_Pesanan dp ON pes.id_pesanan = dp.id_pesanan
JOIN Menu m ON dp.id_menu = m.id_menu;

```

Perintah ini membuat tabel virtual View_Struk_Pesanan yang di dalamnya mengombinasikan empat tabel relasional sekaligus, yaitu Pesanan, Pelanggan, Detail_Pesanan, dan Menu melalui teknik multi-join berbasis kesamaan kolom key (primary key dan foreign key). Objek ini merangkum data krusial struk belanja mulai dari nomor nota, nama pembeli, nama hidangan, kuantitas porsi, hingga kalkulasi biayanya. Melalui pemanggilan `SELECT * FROM View_Struk_Pesanan;`, pengembang sistem di bagian front-end aplikasi dapat langsung menarik rincian data struk yang lengkap tanpa perlu menulis ulang baris kode join yang panjang di sisi program aplikasi.

C. Stored Procedure

1. Deskripsi Logika

Stored procedure adalah kumpulan perintah sql yang disimpan di dalam database sebagai satu kesatuan. Dalam basis data, konsep stored procedure memiliki konsep yang sama dengan pendefinisian fungsi dalam pemrograman.

- a. Kegunaan: Mengelompokkan serangkaian query yang sering dijalankan agar bisa dieksekusi cukup dengan satu perintah panggilan (CALL).
- b. Keunggulan: Meningkatkan performa (karena sudah dikompilasi oleh database) dan meningkatkan keamanan (administrator bisa memberi hak akses ke prosedur saja, tanpa memberikan akses langsung ke tabel).

2. Script dan Contoh Eksekusi

Contoh eksekusi dilakukan untuk menghitung total pesanan dalam data restoran. Parameter yang digunakan adalah kolom “id_pesanan”. Berikut adalah query yang digunakan untuk menghitung total pesanan :

```

DELIMITER //
CREATE PROCEDURE HitungTotalPesanan(IN p_id_pesanan VARCHAR(50))
BEGIN
    DECLARE v_total_keseluruhan DECIMAL(10,2);

```

```

        SELECT SUM(subtotal) INTO v_total_keseluruhan FROM Detail_Pesanan
WHERE id_pesanan = p_id_pesanan;
        UPDATE Pesanan SET total_bayar = IFNULL(v_total_keseluruhan, 0)
WHERE id_pesanan = p_id_pesanan;
END //
DELIMITER ;

```

Query ini membuat tabel pesanan secara otomatis berdasarkan id_pesanan yang berada pada tabel Detail_Pesanan. Setiap pesanan yang berada di kolom detail pesanan akan dijumlahkan dan nilainya dimasukkan pada kolom total_keseluruhan. Perintah IFNULL() digunakan ketika tidak ada pesanan yang diproses atau sedang pending. Peran IFNULL() dalam *query* ini sangat krusial, karena mencegah masuk nilai NULL.

D. Trigger

1. Deskripsi Logika

Trigger adalah prosedur khusus yang dijalankan secara otomatis oleh database sebagai respon terhadap suatu peristiwa (*event*) tertentu pada tabel, seperti INSERT, UPDATE, atau DELETE.

- a. Kegunaan : Menjaga integritas data (konsistensi) secara otomatis. Trigger adalah cara sistem memastikan aturan bisnis tetap berjalan tanpa perlu intervensi manual setiap kali data berubah.
- b. Keunggulan: Otomatisasi. Anda tidak perlu khawatir lupa memperbarui data terkait (misalnya stok barang) karena database yang akan melakukannya untuk Anda.

2. Script dan Contoh Eksekusi

Contoh eksekusi dilakukan untuk mencatat dan menghitung subtotal hitung otomatis setiap kali ada pesanan baru yang ditambahkan dalam tabel Detail_Pesanan. Berikut adalah query yang digunakan untuk mencatat dan menghitung subtotal hitung otomatis:

```

DELIMITER //
CREATE TRIGGER trg_auto_hitung_subtotal
BEFORE INSERT ON Detail_Pesanan
FOR EACH ROW
BEGIN
    DECLARE v_harga_menu DECIMAL(10,2);
    SELECT harga INTO v_harga_menu FROM Menu WHERE id_menu =
NEW.id_menu;

```



```

        SET NEW.subtotal = NEW.jumlah_porsi * v_harga_menu;
    END //
DELIMITER ;

```

Query ini akan mencatat saat dimasukkan data baru pada tabel Detail_Pesanan. Query TRIGGER akan melihat data yang dimasukkan ke dalam tabel Detail_Pesanan. Sistem akan melihat menu apa yang akan dipesan dan secara otomatis akan mencari harga menu pada tabel harga_menu berdasarkan id_menu. Setelah harganya didapat, sistem akan mengalikan harga satuan tersebut dengan jumlah porsi pesanan. Hasil perkalian tadi langsung dimasukkan secara otomatis ke dalam kolom subtotal. Setelah subtotalnya terisi, barulah seluruh data pesanan tersebut resmi disimpan ke dalam database

E. Function

1. Deskripsi Fungsi

Konsep dasar *function* dalam SQL adalah blok kode yang dirancang khusus untuk melakukan perhitungan atau pemrosesan data, dan wajib mengembalikan satu nilai (*return value*). Yang membedakan konsep function dan stored procedure terletak pada penggunaan query seperti INSERT, UPDATE, dll. Jika stored procedure dapat menggunakan query tersebut, function hanya dapat melakukan perhitungan dan *outputnya* berupa nilai.

- Kegunaan : Menyederhanakan query yang rumit. Jika Anda memiliki rumus hitungan yang sering dipakai (seperti pajak, diskon, atau format teks), gunakan *Function*.
- Keunggulan: Membuat query SQL Anda lebih bersih, mudah dibaca, dan konsisten (rumus hitungan tersentralisasi di satu tempat).

2. Script dan Contoh Penggunaan

Contoh eksekusi FUNCTION berikut ini diterapkan untuk melakukan klasifikasi kategori harga menu berdasarkan besaran nominal harga per item. Berikut adalah query yang digunakan untuk mencatat dan menghitung subtotal hitung otomatis:

```

DELIMITER //
CREATE FUNCTION KlasifikasiHargaMenu(p_harga DECIMAL(10,2))
RETURNS VARCHAR(20)
DETERMINISTIC
BEGIN
    DECLARE v_tier VARCHAR(20);
    p_harga >= 45000 THEN SET v_tier = 'Premium Tier';

```

```

        ELSEIF p_harga >= 25000 THEN SET v_tier = 'Standard Tier';
        ELSE SET v_tier = 'Budget Friendly';
        END IF;
        RETURN v_tier;
END //
DELIMITER ;

```

Query function dapat diterapkan untuk mengklasifikasikan harga menu berdasarkan besaran nominal harga per item. Query tersebut membuat sebuah fungsi `KlasifikasiHargaMenu` dengan parameter `p_harga` yang bekerja dengan cara menerima input berupa nominal harga menu, lalu mengevaluasinya secara berurutan untuk menentukan kategori harga; jika harga menu Rp45.000 ke atas maka dikategorikan sebagai 'Premium Tier', jika harga menu antara Rp25.000 hingga Rp44.999 maka dikategorikan sebagai 'Standard Tier', dan jika harga menu di bawah Rp25.000 akan otomatis masuk sebagai 'Budget Friendly', untuk kemudian fungsi ini mengembalikan label teks tersebut sebagai hasil akhirnya.

BAB VI

PENUTUP

A. Kesimpulan

Berdasarkan hasil perancangan, normalisasi, dan pengujian pada Sistem Manajemen Restoran, dapat ditarik beberapa kesimpulan sebagai berikut:

1. Keberhasilan perancangan dan normalisasi: Basis data relasional telah berhasil dirancang dan dinormalisasi hingga bentuk normal ketiga (3NF). Proses ini secara efektif mengurangi atribut yang terduplikasi, menghilangkan ketergantungan parsial, dan menghapus ketergantungan transitif. Hasilnya, terbentuk sebelas entitas tabel yang terstruktur dengan baik (Pelanggan, Karyawan, Meja, Menu, Bahan_Baku, Menu_Bahan, Pembayaran, Pesanan, Detail_Pesanan, Reservasi, dan Review) yang bebas dari redundansi dan anomali data.
2. Otomasi proses bisnis: Penerapan objek sql terbukti mampu mengotomatisasi dan menyederhanakan proses operasional restoran. Penggunaan trigger berhasil melakukan perhitungan subtotal saat detail pesanan dimasukkan secara otomatis. stored procedure berfungsi efektif dalam mengakumulasi total bayar pada tabel Pesanan. Selain itu, function mempermudah klasifikasi pelanggan berdasarkan total belanja, dan view mempermudah penyajian data laporan harian serta cetak struk.
3. Kinerja sistem yang terintegrasi: Integrasi antara struktur data yang telah dinormalisasi dengan otomasi objek sql di dalam DBMS MySQL Workbench 8.0 CE menghasilkan sistem manajemen basis data yang terintegrasi, efisien, akurat, dan mudah dipelihara. Pengujian fungsional melalui berbagai query (termasuk JOIN, Subquery, CTE, dan Agregasi) menunjukkan bahwa sistem mampu menangani skenario transaksional yang kompleks dengan baik.

B. Saran

Untuk pengembangan dan penyempurnaan sistem basis data manajemen restoran ini di masa mendatang, terdapat beberapa saran yang dapat dipertimbangkan:

1. Pengembangan antarmuka pengguna (*Front-End*): Interaksi data masih dilakukan langsung melalui *script* sql di MySQL Workbench. Disarankan untuk mengembangkan aplikasi antarmuka berbasis *web* atau *mobile* yang terhubung ke

basis data, sehingga staf restoran dan pelanggan dapat mengoperasikan sistem dengan lebih mudah.

2. Penyusunan prosedur pencadangan (*backup*): Untuk menjaga keamanan data operasional restoran dari risiko kehilangan atau kerusakan perangkat keras, perlu dirancang prosedur pencadangan otomatis basis data secara berkala.

LAMPIRAN

Lampiran 1. Script SQL

```
-- =====
-- LANGKAH 1: INISIALISASI DATABASE & PEMBERSIHAN
-- =====

DROP DATABASE IF EXISTS restoran;
CREATE DATABASE IF NOT EXISTS restoran;
USE restoran;

SET FOREIGN_KEY_CHECKS = 0;
DROP TABLE IF EXISTS Review;
DROP TABLE IF EXISTS Detail_Pesanan;
DROP TABLE IF EXISTS Pesanan;
DROP TABLE IF EXISTS Pembayaran;
DROP TABLE IF EXISTS Reservasi;
DROP TABLE IF EXISTS Menu_Bahan;
DROP TABLE IF EXISTS Bahan_Baku;
DROP TABLE IF EXISTS Menu;
DROP TABLE IF EXISTS Meja;
DROP TABLE IF EXISTS Karyawan;
DROP TABLE IF EXISTS Pelanggan;
SET FOREIGN_KEY_CHECKS = 1;

-- =====
-- LANGKAH 2: PEMBUATAN STRUKTUR TABEL (DDL)
-- =====

-- 1. Tabel Pelanggan
CREATE TABLE Pelanggan (
    id_pelanggan VARCHAR(10) PRIMARY KEY,
    nama_pelanggan VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE,
    no_telp VARCHAR(15),
    alamat TEXT
);

-- 2. Tabel Karyawan
```

```

CREATE TABLE Karyawan (
    id_karyawan VARCHAR(10) PRIMARY KEY,
    nama_karyawan VARCHAR(100) NOT NULL,
    jabatan VARCHAR(50) NOT NULL,
    shift VARCHAR(20),
    nomor_telepon VARCHAR(15)
);

-- 3. Tabel Meja
CREATE TABLE Meja (
    nomor_meja VARCHAR(10) PRIMARY KEY,
    kapasitas INT NOT NULL CHECK (kapasitas > 0)
);

-- 4. Tabel Menu
CREATE TABLE Menu (
    id_menu VARCHAR(10) PRIMARY KEY,
    nama_menu VARCHAR(100) NOT NULL,
    harga DECIMAL(10, 2) NOT NULL CHECK (harga >= 0)
);

-- 5. Tabel Pembayaran
CREATE TABLE Pembayaran (
    id_bayar VARCHAR(10) PRIMARY KEY,
    metode_pembayaran VARCHAR(30) NOT NULL,
    status_bayar VARCHAR(20) NOT NULL
);

-- 6. Tabel Reservasi
CREATE TABLE Reservasi (
    id_reservasi VARCHAR(10) PRIMARY KEY,
    id_pelanggan VARCHAR(10),
    nomor_meja VARCHAR(10),
    tanggal_reservasi DATE NOT NULL,
    jumlah_tamu INT NOT NULL CHECK (jumlah_tamu > 0),
    status_reservasi VARCHAR(20) DEFAULT 'Pending',
    FOREIGN KEY (id_pelanggan) REFERENCES Pelanggan(id_pelanggan),
    FOREIGN KEY (nomor_meja) REFERENCES Meja(nomor_meja)
);

```

-- 7. Tabel Pesanan

```
CREATE TABLE Pesanan (  
    id_pesanan VARCHAR(10) PRIMARY KEY,  
    id_pelanggan VARCHAR(10),  
    id_reservasi VARCHAR(10),  
    id_karyawan VARCHAR(10),  
    id_bayar VARCHAR(10),  
    nomor_meja VARCHAR(10),  
    tanggal_pemesanan DATE NOT NULL,  
    total_bayar DECIMAL(10, 2) DEFAULT 0.00,  
    FOREIGN KEY (id_pelanggan) REFERENCES Pelanggan(id_pelanggan),  
    FOREIGN KEY (id_karyawan) REFERENCES Karyawan(id_karyawan),  
    FOREIGN KEY (id_reservasi) REFERENCES Reservasi(id_reservasi),  
    FOREIGN KEY (nomor_meja) REFERENCES Meja(nomor_meja) ON DELETE SET  
NULL,  
    FOREIGN KEY (id_bayar) REFERENCES Pembayaran(id_bayar)  
);
```

-- 8. Tabel Detail Pesanan

```
CREATE TABLE Detail_Pesanan (  
    id_detail_pesanan VARCHAR(10) PRIMARY KEY,  
    id_pesanan VARCHAR(10),  
    id_menu VARCHAR(10),  
    jumlah_porsi INT NOT NULL CHECK (jumlah_porsi >= 1),  
    subtotal DECIMAL(10, 2) NOT NULL DEFAULT 0.00,  
    catatan_khusus TEXT,  
    FOREIGN KEY (id_pesanan) REFERENCES Pesanan(id_pesanan) ON DELETE  
CASCADE,  
    FOREIGN KEY (id_menu) REFERENCES Menu(id_menu)  
);
```

-- 9. Tabel Review

```
CREATE TABLE Review (  
    id_Review VARCHAR(10) PRIMARY KEY,  
    id_pesanan VARCHAR(10),  
    rating INT NOT NULL CHECK (rating BETWEEN 1 AND 5),  
    komentar TEXT,
```

```

        FOREIGN KEY (id_pesanan) REFERENCES Pesanan(id_pesanan) ON DELETE
CASCADE
);

-- 10. Tabel Bahan Baku
CREATE TABLE Bahan_Baku (
    id_bahan VARCHAR(10) PRIMARY KEY,
    nama_bahan VARCHAR(100) NOT NULL
);

-- 11. Tabel Menu Bahan (Many-to-Many)
CREATE TABLE Menu_Bahan (
    id_menu VARCHAR(10),
    id_bahan VARCHAR(10),
    PRIMARY KEY (id_menu, id_bahan),
    FOREIGN KEY (id_menu) REFERENCES Menu(id_menu) ON DELETE CASCADE,
    FOREIGN KEY (id_bahan) REFERENCES Bahan_Baku(id_bahan) ON DELETE
CASCADE
);

-- =====
-- LANGKAH 3: PROGRAM DATABASE (FUNCTION, TRIGGER, SP, & VIEW)
-- =====

-- A. FUNCTION
DELIMITER //
CREATE FUNCTION KlasifikasiHargaMenu(p_harga DECIMAL(10,2))
RETURNS VARCHAR(20)
DETERMINISTIC
BEGIN
    DECLARE v_tier VARCHAR(20);
    IF p_harga >= 45000 THEN SET v_tier = 'Premium Tier';
    ELSEIF p_harga >= 25000 THEN SET v_tier = 'Standard Tier';
    ELSE SET v_tier = 'Budget Friendly';
    END IF;
    RETURN v_tier;
END //
DELIMITER ;

```



```

-- B. TRIGGER
DELIMITER //
CREATE TRIGGER trg_auto_hitung_subtotal
BEFORE INSERT ON Detail_Pesanan
FOR EACH ROW
BEGIN
    DECLARE v_harga_menu DECIMAL(10,2);
    SELECT harga INTO v_harga_menu FROM Menu WHERE id_menu = NEW.id_menu;
    SET NEW.subtotal = NEW.jumlah_porsi * v_harga_menu;
END //
DELIMITER ;

-- C. STORED PROCEDURE
DELIMITER //
CREATE PROCEDURE HitungTotalPesanan(IN p_id_pesanan VARCHAR(10))
BEGIN
    DECLARE v_total_keseluruhan DECIMAL(10,2);
    SELECT SUM(subtotal) INTO v_total_keseluruhan FROM Detail_Pesanan
WHERE id_pesanan = p_id_pesanan;
    UPDATE Pesanan SET total_bayar = IFNULL(v_total_keseluruhan, 0)
WHERE id_pesanan = p_id_pesanan;
END //
DELIMITER ;

-- D. VIEW SISTEM LAPORAN
-- VIEW 1: Laporan Pendapatan Harian Restoran
CREATE VIEW View_Pendapatan_Harian AS
SELECT
    tanggal_pemesanan AS tanggal_transaksi,
    COUNT(DISTINCT id_pesanan) AS jumlah_transaksi,
    SUM(total_bayar) AS total_pendapatan
FROM Pesanan
GROUP BY tanggal_pemesanan;

-- MENAMPILKAN DATA DARI PROGRAM VIEW --

```

```

SELECT * FROM View_Pendapatan_Harian;

-- VIEW 2: Cetak Struk Transaksi Detail Lengkap Pelanggan
CREATE VIEW View_Struk_Pesanan AS
SELECT
    pes.id_pesanan, p.nama_pelanggan, m.nama_menu, dp.jumlah_porsi,
    m.harga AS harga_satuan, dp.subtotal AS subtotal
FROM Pesanan pes
JOIN Pelanggan p ON pes.id_pelanggan = p.id_pelanggan
JOIN Detail_Pesanan dp ON pes.id_pesanan = dp.id_pesanan
JOIN Menu m ON dp.id_menu = m.id_menu;

-- MENAMPILKAN DATA DARI PROGRAM VIEW ---
SELECT * FROM View_Struk_Pesanan;

-- =====
-- LANGKAH 4: DATA MANIPULATION LANGUAGE (DML) - INSERT DATA
-- =====

-- 1. PELANGGAN
INSERT INTO Pelanggan (id_pelanggan, nama_pelanggan, email, no_telp,
alamat) VALUES
('P001', 'andi', 'andi@email.com', '8267354890', 'Jakarta'),
('P002', 'budi', 'budi@email.com', '8987765356', 'Bandung'),
('P003', 'citra', 'citra@email.com', '8113456789', 'Surabaya'),
('P004', 'dewi', 'dewi@email.com', '8114567890', 'Semarang'),
('P005', 'eko', 'eko@email.com', '8115678901', 'Yogyakarta'),
('P006', 'fani', 'fani@email.com', '8116789012', 'Malang'),
('P007', 'gita', 'gita@email.com', '8117890123', 'Solo'),
('P008', 'hadi', 'hadi@email.com', '8118901234', 'Bogor'),
('P009', 'indah', 'indah@email.com', '8119012345', 'Bekasi'),
('P010', 'joko', 'joko@email.com', '8120123456', 'Depok'),
('P011', 'kurnia', 'kurnia@email.com', '8221234567', 'Tangerang'),
('P012', 'laras', 'laras@email.com', '8122345678', 'Cirebon'),
('P013', 'maman', 'maman@email.com', '8123456789', 'Garut'),
('P014', 'nina', 'nina@email.com', '8124567890', 'Tasik'),
('P015', 'oki', 'oki@email.com', '8125678901', 'Sukabumi'),

```

```
( 'P016', 'prima', 'prima@email.com', '8126789012', 'Madura'),
( 'P017', 'rani', 'rani@email.com', '8127890123', 'Jember'),
( 'P018', 'sinta', 'sinta@email.com', '8128901234', 'Pasuruan'),
( 'P019', 'taufik', 'taufik@email.com', '8929012345', 'Madiun'),
( 'P020', 'vika', 'vika@email.com', '8130123456', 'Kediri'),
( 'P021', 'wisnu', 'wisnu@email.com', '8131234567', 'Blitar'),
( 'P022', 'yuda', 'yuda@email.com', '8232345678', 'Denpasar'),
( 'P023', 'zain', 'zain@email.com', '8133456789', 'Makassar'),
( 'P024', 'amalia', 'amalia@email.com', '8134567890', 'Medan'),
( 'P025', 'cinta', 'cinta@email.com', '8908354678', 'Bandung'),
( 'P026', 'ken', 'ken@email.com', '8214673890', 'Majalengka'),
( 'P027', 'rio', 'rio@email.com', '8247804576', 'Bogor'),
( 'P028', 'lisa', 'lisa@email.com', '08138901234', 'Purwokerto'),
( 'P029', 'fredo', 'fredo@email.com', '08139012345', 'Purwokerto'),
( 'P030', 'noval', 'noval@email.com', '08140123456', 'Cilacap'),
( 'P031', 'tari', 'tari@email.com', '08141234567', 'Banyumas'),
( 'P032', 'bagas', 'bagas@email.com', '08142345678', 'Kebumen'),
( 'P033', 'dina', 'dina@email.com', '08143456789', 'Wonosobo'),
( 'P034', 'hendra', 'hendra@email.com', '08144567890', 'Banjarnegara');
```

-- 2. KARYAWAN

```
INSERT INTO Karyawan (id_karyawan, nama_karyawan, jabatan, shift,
nomor_telepon) VALUES
```

```
( 'K01', 'martin', 'Manajer', '-', '8924687243'),
( 'K02', 'adi', 'Waiter', 'Pagi', '8222333444'),
( 'K03', 'mina', 'Waiter', 'Sore', '8223456789'),
( 'K04', 'reno', 'Chef', 'Pagi', '8224567890'),
( 'K05', 'maya', 'Kasir', 'Pagi', '8225678901'),
( 'K06', 'hana', 'Chef', 'Sore', '89167338902'),
( 'K07', 'aji', 'Kasir', 'Sore', '8959012758'),
( 'K08', 'ayu', 'Pencuci Piring', 'Pagi', '894237889'),
( 'K09', 'siti', 'Pencuci Piring', 'Sore', '8224773905'),
( 'K10', 'supriyadi', 'OB', 'Pagi', '8246637441'),
( 'K11', 'rudi', 'OB', 'Sore', '8567223665'),
( 'K12', 'nana', 'Waiter', 'Siang', '8563390547'),
( 'K13', 'galih', 'Waiter', 'Pagi', '8367574411'),
( 'K14', 'albert', 'Chef', 'Siang', '8360012789'),
( 'K15', 'arnold', 'Chef', 'Pagi', '8874419023'),
( 'K16', 'winato', 'OB', 'Siang', '8864783991'),
```

```

('K17', 'sasuke', 'Kasir', 'Siang', '89433671122'),
('K18', 'tedi', 'Waiter', 'Siang', '89001343376'),
('K19', 'dion', 'Waiter', 'Sore', '89033788122'),
('K20', 'juna', 'Chef', 'Siang', '8244780361'),
('K21', 'renata', 'Chef', 'Sore', '8337886520');

```

-- 3. MEJA

```

INSERT INTO Meja (nomor_meja, kapasitas) VALUES
('T01', 6), ('T02', 2), ('T03', 2), ('T04', 4), ('T05', 4),
('T06', 2), ('T07', 2), ('T08', 4), ('T09', 2);

```

-- 4. MENU

```

INSERT INTO Menu (id_menu, nama_menu, harga) VALUES
('M01', 'Nasi Goreng', 35000),
('M02', 'Es Teh', 15000),
('M03', 'Ayam Bakar', 45000),
('M04', 'Mie Goreng', 30000),
('M05', 'Jus Alpukat', 20000),
('M06', 'Mie Ayam', 30000),
('M07', 'Jus Mangga', 20000),
('M08', 'Ayam Madu', 38000),
('M09', 'Gurame Bakar', 45000),
('M10', 'Seafood Siram', 85000),
('M11', 'Ayam Bali', 40000),
('M12', 'Dubai Chewy Cookie', 50000),
('M13', 'Es Buah', 22000),
('M14', 'Ketoprak', 20000),
('M15', 'Jus Sirsak', 18000),
('M16', 'Air Mineral', 10000),
('M17', 'Cheesecake', 35000),
('M18', 'Bebek Goreng', 50000),
('M19', 'Mendoan', 10000),
('M20', 'Es Pisang Ijo', 17000);

```

-- 5. MASTER PEMBAYARAN

```

INSERT INTO Pembayaran (id_bayar, metode_pembayaran, status_bayar)
VALUES
('PY01', 'Tunai', 'Lunas'),
('PY02', 'QRIS', 'Lunas'),

```

```
('PY03', 'Kartu', 'Lunas'),  
('PY04', 'Transfer', 'Lunas');
```

```
-- 6. BAHAN BAKU
```

```
INSERT INTO Bahan_Baku (id_bahan, nama_bahan) VALUES
```

```
('B01', 'Beras'),  
('B02', 'Kecap'),  
('B03', 'Bawang'),  
('B04', 'Telur'),  
('B05', 'Teh'),  
('B06', 'Gula'),  
('B07', 'Garam'),  
('B08', 'Cabai'),  
('B09', 'Mentega'),  
('B10', 'Minyak Goreng'),  
('B11', 'Mie'),  
('B12', 'Sawi'),  
('B13', 'Kol'),  
('B14', 'Daun Bawang'),  
('B15', 'Timun'),  
('B16', 'Alpukat'),  
('B17', 'Mangga'),  
('B18', 'Madu'),  
('B19', 'Ayam'),  
('B20', 'Gurame'),  
('B21', 'Udang'),  
('B22', 'Kepiting'),  
('B23', 'Cumi'),  
('B24', 'Kerang'),  
('B25', 'Jagung'),  
('B26', 'Saus'),  
('B27', 'Coklat'),  
('B28', 'Kunafa'),  
('B29', 'Pistachio'),  
('B30', 'Marshmellow'),  
('B31', 'Melon'),  
('B32', 'Buah Naga'),  
('B33', 'Pir'),  
('B34', 'Es Batu'),
```

```

('B35', 'Kelapa'),
('B36', 'Sirup'),
('B37', 'Bihun'),
('B38', 'Kacang'),
('B39', 'Toge'),
('B40', 'Sirsak'),
('B41', 'Susu Kental Manis'),
('B42', 'Air'),
('B43', 'Keju'),
('B44', 'Tepung'),
('B45', 'Bebek'),
('B46', 'Tempe'),
('B47', 'Pisang'),
('B48', 'Santan');

```

-- 7. MENU BAHAN

```

INSERT INTO Menu_Bahan (id_menu, id_bahan) VALUES
('M01', 'B01'), ('M01', 'B04'), ('M01', 'B03'), ('M01', 'B02'), ('M01',
'B10'),
('M02', 'B05'), ('M02', 'B34'),
('M03', 'B19'), ('M03', 'B08'), ('M03', 'B03'), ('M03', 'B02'), ('M03',
'B01'),
('M04', 'B11'), ('M04', 'B04'), ('M04', 'B03'), ('M04', 'B02'), ('M04',
'B10'),
('M05', 'B16'), ('M05', 'B06'),
('M06', 'B11'), ('M06', 'B19'), ('M06', 'B12'), ('M06', 'B03'),
('M07', 'B17'), ('M07', 'B06'), ('M07', 'B34'),
('M08', 'B19'), ('M08', 'B18'), ('M08', 'B03'), ('M08', 'B10'),
('M09', 'B20'), ('M09', 'B08'), ('M09', 'B03'), ('M09', 'B02'),
('M10', 'B21'), ('M10', 'B22'), ('M10', 'B23'), ('M10', 'B24'), ('M10',
'B26'),
('M11', 'B19'), ('M11', 'B01'), ('M11', 'B08'), ('M11', 'B10'),
('M12', 'B27'), ('M12', 'B28'), ('M12', 'B29'), ('M12', 'B30'),
('M13', 'B31'), ('M13', 'B32'), ('M13', 'B33'), ('M13', 'B34'), ('M13',
'B36'),
('M14', 'B15'), ('M14', 'B37'), ('M14', 'B38'), ('M14', 'B39'),
('M15', 'B40'), ('M15', 'B06'), ('M15', 'B34'),
('M16', 'B42'),
('M17', 'B43'), ('M17', 'B41'), ('M17', 'B06'),

```

```

('M18', 'B45'), ('M18', 'B10'), ('M18', 'B01'),
('M19', 'B46'), ('M19', 'B44'), ('M19', 'B07'), ('M19', 'B10'),
('M20', 'B47'), ('M20', 'B36'), ('M20', 'B34'), ('M20', 'B48');

-- 8. RESERVASI
INSERT INTO Reservasi (id_reservasi, id_pelanggan, nomor_meja,
tanggal_reservasi, jumlah_tamu, status_reservasi) VALUES
('RS01', 'P001', 'T05', '2026-01-10', 2, 'Selesai'),
('RS02', 'P003', 'T01', '2026-01-11', 5, 'Selesai'),
('RS03', 'P005', 'T02', '2026-01-12', 2, 'Selesai'),
('RS04', 'P007', 'T06', '2026-01-13', 4, 'Selesai'),
('RS05', 'P009', 'T05', '2026-01-13', 2, 'Selesai'),
('RS06', 'P011', 'T01', '2026-01-14', 6, 'Selesai'),
('RS07', 'P014', 'T08', '2026-01-14', 4, DEFAULT),
('RS08', 'P017', 'T05', '2026-01-15', 2, 'Selesai'),
('RS09', 'P018', 'T09', '2026-01-15', 2, DEFAULT),
('RS10', 'P021', 'T01', '2026-01-15', 4, 'Selesai'),
('RS11', 'P023', 'T03', '2026-01-14', 2, 'Selesai'),
('RS12', 'P027', 'T06', '2026-01-17', 3, 'Selesai'),
('RS13', 'P011', 'T01', '2026-01-18', 2, 'Selesai'),
('RS14', 'P029', 'T08', '2026-01-19', 3, 'Selesai'),
('RS15', 'P031', 'T04', '2026-01-20', 4, 'Selesai'),
('RS16', 'P033', 'T01', '2026-01-21', 5, 'Selesai'),
('RS17', 'P007', 'T05', '2026-01-22', 2, 'Selesai');

-- 9. PESANAN
INSERT INTO Pesanan (id_pesanan, id_pelanggan, id_reservasi,
id_karyawan, id_bayar, nomor_meja, tanggal_pemesanan, total_bayar)
VALUES
('PS01', 'P001', 'RS01', 'K02', 'PY01', 'T05', '2026-01-10', 0.00),
('PS02', 'P002', NULL, 'K12', 'PY02', 'T02', '2026-01-10', 0.00),
('PS03', 'P003', 'RS02', 'K13', 'PY03', 'T01', '2026-01-11', 0.00),
('PS04', 'P004', NULL, 'K02', 'PY04', 'T04', '2026-01-11', 0.00),
('PS05', 'P005', 'RS03', 'K12', 'PY02', 'T02', '2026-01-12', 0.00),
('PS06', 'P006', NULL, 'K13', 'PY01', 'T03', '2026-01-12', 0.00),
('PS07', 'P007', 'RS04', 'K02', 'PY01', 'T06', '2026-01-13', 0.00),
('PS08', 'P008', NULL, 'K12', 'PY02', 'T04', '2026-01-13', 0.00),
('PS09', 'P009', 'RS05', 'K03', 'PY01', 'T05', '2026-01-13', 0.00),
('PS10', 'P010', NULL, 'K13', 'PY02', 'T02', '2026-01-14', 0.00),

```

```

('PS11', 'P011', 'RS06', 'K02', 'PY03', 'T01', '2026-01-14', 0.00),
('PS12', 'P012', NULL, 'K12', 'PY01', 'T07', '2026-01-14', 0.00),
('PS13', 'P013', NULL, 'K19', 'PY02', 'T04', '2026-01-14', 0.00),
('PS14', 'P015', NULL, 'K02', 'PY01', 'T03', '2026-01-15', 0.00),
('PS15', 'P016', NULL, 'K12', 'PY02', 'T02', '2026-01-15', 0.00),
('PS16', 'P017', 'RS08', 'K13', 'PY04', 'T05', '2026-01-15', 0.00),
('PS17', 'P019', NULL, 'K03', 'PY01', 'T06', '2026-01-15', 0.00),
('PS18', 'P020', NULL, 'K18', 'PY01', 'T04', '2026-01-15', 0.00),
('PS19', 'P021', 'RS10', 'K19', 'PY03', 'T01', '2026-01-15', 0.00),
('PS20', 'P001', NULL, 'K18', 'PY02', 'T02', '2026-01-11', 0.00),
('PS21', 'P002', NULL, 'K18', 'PY01', 'T05', '2026-01-13', 0.00),
('PS22', 'P022', NULL, 'K19', 'PY02', 'T04', '2026-01-12', 0.00),
('PS23', 'P023', 'RS11', 'K03', 'PY01', 'T03', '2026-01-14', 0.00),
('PS24', 'P005', NULL, 'K18', 'PY01', 'T02', '2026-01-14', 0.00),
('PS25', 'P024', NULL, 'K03', 'PY03', 'T05', '2026-01-10', 0.00),
('PS26', 'P025', NULL, 'K02', 'PY01', 'T02', '2026-01-16', 0.00),
('PS27', 'P026', NULL, 'K03', 'PY02', 'T07', '2026-01-16', 0.00),
('PS28', 'P027', 'RS12', 'K12', 'PY03', 'T06', '2026-01-17', 0.00),
('PS29', 'P008', NULL, 'K18', 'PY01', 'T04', '2026-01-17', 0.00),
('PS30', 'P011', 'RS13', 'K13', 'PY02', 'T01', '2026-01-18', 0.00),
('PS31', 'P013', NULL, 'K19', 'PY04', 'T06', '2026-01-18', 0.00),
('PS32', 'P028', NULL, 'K02', 'PY02', 'T03', '2026-01-19', 0.00),
('PS33', 'P029', 'RS14', 'K12', 'PY03', 'T08', '2026-01-19', 0.00),
('PS34', 'P030', NULL, 'K03', 'PY01', 'T02', '2026-01-19', 0.00),
('PS35', 'P016', NULL, 'K13', 'PY02', 'T05', '2026-01-20', 0.00),
('PS36', 'P031', 'RS15', 'K02', 'PY01', 'T04', '2026-01-20', 0.00),
('PS37', 'P032', NULL, 'K18', 'PY01', 'T07', '2026-01-20', 0.00),
('PS38', 'P033', 'RS16', 'K12', 'PY03', 'T01', '2026-01-21', 0.00),
('PS39', 'P034', NULL, 'K19', 'PY02', 'T03', '2026-01-21', 0.00),
('PS40', 'P007', 'RS17', 'K13', 'PY01', 'T05', '2026-01-22', 0.00);

```

```
-- 10. DETAIL PESANAN
```

```

INSERT INTO Detail_Pesanan (id_detail_pesanan, id_pesanan, id_menu,
jumlah_porsi, subtotal, catatan_khusus) VALUES
('D01', 'PS01', 'M01', 2, 0, 'Pedas'),
('D02', 'PS01', 'M02', 1, 0, 'Kurang manis'),
('D03', 'PS02', 'M03', 1, 0, 'Bagian dada'),
('D04', 'PS03', 'M01', 3, 0, 'Tidak pedas'),
('D05', 'PS03', 'M02', 1, 0, 'Biasa'),

```


('D06', 'PS04', 'M04', 1, 0, 'Karet pisah'),
 ('D07', 'PS05', 'M03', 2, 0, 'Pedas manis'),
 ('D08', 'PS06', 'M05', 1, 0, 'Tanpa susu'),
 ('D09', 'PS06', 'M01', 1, 0, 'Sedang'),
 ('D10', 'PS07', 'M03', 1, 0, 'Minta matang'),
 ('D11', 'PS07', 'M02', 1, 0, 'Es banyak'),
 ('D12', 'PS08', 'M04', 1, 0, 'Pedas mampus'),
 ('D13', 'PS09', 'M01', 1, 0, 'Tidak pedas'),
 ('D14', 'PS09', 'M02', 1, 0, 'Manis'),
 ('D15', 'PS10', 'M03', 2, 0, '-'),
 ('D16', 'PS10', 'M02', 1, 0, '-'),
 ('D17', 'PS11', 'M04', 3, 0, '-'),
 ('D18', 'PS11', 'M05', 1, 0, 'Kurang gula'),
 ('D19', 'PS12', 'M01', 1, 0, 'Pedas'),
 ('D20', 'PS13', 'M05', 1, 0, 'Banyakin es'),
 ('D21', 'PS14', 'M02', 1, 0, '-'),
 ('D22', 'PS15', 'M01', 2, 0, 'Pedas'),
 ('D23', 'PS16', 'M04', 1, 0, '-'),
 ('D24', 'PS16', 'M02', 1, 0, '-'),
 ('D25', 'PS17', 'M01', 1, 0, 'Sedang'),
 ('D26', 'PS17', 'M02', 1, 0, '-'),
 ('D27', 'PS18', 'M04', 1, 0, '-'),
 ('D28', 'PS19', 'M01', 1, 0, 'Pedas'),
 ('D29', 'PS20', 'M03', 1, 0, '-'),
 ('D30', 'PS20', 'M02', 1, 0, '-'),
 ('D31', 'PS21', 'M01', 1, 0, 'Sedang'),
 ('D32', 'PS22', 'M18', 1, 0, '-'),
 ('D33', 'PS22', 'M05', 1, 0, 'Kurang gula'),
 ('D34', 'PS23', 'M01', 1, 0, 'Ekstra Pedas'),
 ('D35', 'PS24', 'M02', 1, 0, 'Manis'),
 ('D36', 'PS25', 'M01', 1, 0, 'Tidak Pedas'),
 ('D37', 'PS25', 'M02', 2, 0, 'Tanpa gula'),
 ('D38', 'PS26', 'M03', 1, 0, 'Pedas sekali'),
 ('D39', 'PS26', 'M02', 1, 0, 'Es biasa'),
 ('D40', 'PS27', 'M04', 1, 0, '-'),
 ('D41', 'PS28', 'M05', 1, 0, 'Manis'),
 ('D42', 'PS28', 'M01', 1, 0, 'Tanpa es'),
 ('D43', 'PS29', 'M04', 1, 0, 'Pedas'),
 ('D44', 'PS29', 'M02', 1, 0, 'Kurang mateng'),

```

('D45', 'PS30', 'M01', 1, 0, 'Sedang'),
('D46', 'PS30', 'M05', 1, 0, 'Kurang gula'),
('D47', 'PS31', 'M03', 1, 0, 'Pedas'),
('D48', 'PS31', 'M02', 1, 0, 'Es biasa'),
('D49', 'PS32', 'M01', 1, 0, '-'),
('D50', 'PS33', 'M03', 2, 0, 'Ekstra pedas'),
('D51', 'PS33', 'M05', 1, 0, 'Kurang manis'),
('D52', 'PS34', 'M04', 1, 0, '-'),
('D53', 'PS35', 'M01', 1, 0, 'Sedang'),
('D54', 'PS35', 'M02', 1, 0, '-'),
('D55', 'PS36', 'M03', 1, 0, 'Matang sempurna'),
('D56', 'PS36', 'M04', 1, 0, '-'),
('D57', 'PS37', 'M05', 1, 0, 'Kurang gula'),
('D58', 'PS38', 'M03', 1, 0, 'Pedas'),
('D59', 'PS38', 'M01', 1, 0, '-'),
('D60', 'PS38', 'M02', 1, 0, '-'),
('D61', 'PS39', 'M04', 1, 0, 'Tidak pedas'),
('D62', 'PS40', 'M05', 1, 0, 'Manis'),
('D63', 'PS40', 'M01', 1, 0, 'Biasa');

```

```

-- SINKRONISASI MASSAL NILAI TOTAL_BAYAR DI TABEL PESANAN (PS01 - PS40)
CALL HitungTotalPesanan('PS01'); CALL HitungTotalPesanan('PS02'); CALL
HitungTotalPesanan('PS03');
CALL HitungTotalPesanan('PS04'); CALL HitungTotalPesanan('PS05'); CALL
HitungTotalPesanan('PS06');
CALL HitungTotalPesanan('PS07'); CALL HitungTotalPesanan('PS08'); CALL
HitungTotalPesanan('PS09');
CALL HitungTotalPesanan('PS10'); CALL HitungTotalPesanan('PS11'); CALL
HitungTotalPesanan('PS12');
CALL HitungTotalPesanan('PS13'); CALL HitungTotalPesanan('PS14'); CALL
HitungTotalPesanan('PS15');
CALL HitungTotalPesanan('PS16'); CALL HitungTotalPesanan('PS17'); CALL
HitungTotalPesanan('PS18');
CALL HitungTotalPesanan('PS19'); CALL HitungTotalPesanan('PS20'); CALL
HitungTotalPesanan('PS21');
CALL HitungTotalPesanan('PS22'); CALL HitungTotalPesanan('PS23'); CALL
HitungTotalPesanan('PS24');
CALL HitungTotalPesanan('PS25'); CALL HitungTotalPesanan('PS26'); CALL
HitungTotalPesanan('PS27');

```

```
CALL HitungTotalPesanan('PS28'); CALL HitungTotalPesanan('PS29'); CALL
HitungTotalPesanan('PS30');
CALL HitungTotalPesanan('PS31'); CALL HitungTotalPesanan('PS32'); CALL
HitungTotalPesanan('PS33');
CALL HitungTotalPesanan('PS34'); CALL HitungTotalPesanan('PS35'); CALL
HitungTotalPesanan('PS36');
CALL HitungTotalPesanan('PS37'); CALL HitungTotalPesanan('PS38'); CALL
HitungTotalPesanan('PS39');
CALL HitungTotalPesanan('PS40');
```

```
-- 11. REVIEW
```

```
INSERT INTO Review (id_Review, id_pesanan, rating, komentar) VALUES
('R001', 'PS01', 5, 'Enak sekali'),
('R002', 'PS02', 4, 'Pelayanan bagus'),
('R003', 'PS04', 3, 'Agak lama'),
('R004', 'PS05', 5, 'Mantap bumbunya'),
('R005', 'PS07', 2, 'Ayam kurang matang'),
('R006', 'PS09', 4, 'Tempat bersih'),
('R007', 'PS11', 5, 'Pelayanan ramah'),
('R008', 'PS13', 4, 'Jus segar'),
('R009', 'PS14', 1, 'Meja kotor'),
('R010', 'PS16', 5, 'Recommended!'),
('R011', 'PS18', 4, 'Oke lah'),
('R012', 'PS22', 5, 'Pelayanan cepat'),
('R013', 'PS25', 4, 'Suasana nyaman'),
('R014', 'PS26', 3, 'Ayamnya agak keras'),
('R015', 'PS28', 4, 'Minumannya enak'),
('R016', 'PS30', 5, 'Selalu enak'),
('R017', 'PS31', 3, 'Lumayan'),
('R018', 'PS33', 4, 'Porsinya banyak'),
('R019', 'PS35', 5, 'Makanan enak banget'),
('R020', 'PS36', 5, 'Chef-nya handal'),
('R021', 'PS38', 4, 'Lengkap dan enak'),
('R022', 'PS40', 4, 'Tempatnya nyaman');
```

```
-- =====
-- LANGKAH 5: SINTAKS MENAMPILKAN SEMUA HASIL TABEL & PROGRAM DATABASE
-- =====
```

```

--
-- --- MENAMPILKAN DATA DARI SEMUA 11 TABEL MASTER & TRANSAKSI ---

SELECT * FROM Pelanggan;
SELECT * FROM Karyawan;
SELECT * FROM Meja;
SELECT * FROM Menu;
SELECT * FROM Pembayaran;
SELECT * FROM Reservasi;
SELECT * FROM Pesanan;
SELECT * FROM Detail_Pesanan;
SELECT * FROM Review;
SELECT * FROM Bahan_Baku;
SELECT * FROM Menu_Bahan;

-- --- MENAMPILKAN HASIL PENGUJIAN 10 QUERY SISTEM ---

-- [KELOMPOK A: 3 QUERY SEDERHANA]
-- 1. Menampilkan menu beserta kelas tiering harganya menggunakan
FUNCTION
SELECT id_menu, nama_menu, harga, KlasifikasiHargaMenu(harga) AS
tier_harga
FROM Menu;

-- 2. Menampilkan daftar meja yang memiliki kapasitas 3 orang atau lebih
SELECT nomor_meja, kapasitas FROM Meja
WHERE kapasitas >= 3;

-- 3. Tampilkan Karyawan & Jabatan
SELECT nama_karyawan, jabatan FROM Karyawan;

-- [KELOMPOK B: 4 QUERY JOIN MINIMAL 3 TABEL]
-- 4. JOIN 3 Tabel: Pesanan, Pelanggan, dan Karyawan
SELECT pes.id_pesanan, pes.tanggal_pemesanan, p.nama_pelanggan,
k.nama_karyawan
FROM Pesanan pes
JOIN Pelanggan p ON pes.id_pelanggan = p.id_pelanggan
JOIN Karyawan k ON pes.id_karyawan = k.id_karyawan;

```

```
-- 5. JOIN 3 Tabel: Detail Pesanan, Pesanan, dan Menu
SELECT dp.id_detail_pesanan, pes.tanggal_pemesanan, m.nama_menu,
dp.jumlah_porsi, dp.subtotal
FROM Detail_Pesanan dp
JOIN Pesanan pes ON dp.id_pesanan = pes.id_pesanan
JOIN Menu m ON dp.id_menu = m.id_menu;
```

```
-- 6. JOIN 3 Tabel: Menu, Menu_Bahan, dan Bahan_Baku
SELECT m.nama_menu, bb.nama_bahan
FROM Menu m
JOIN Menu_Bahan mb ON m.id_menu = mb.id_menu
JOIN Bahan_Baku bb ON mb.id_bahan = bb.id_bahan;
```

```
-- 7. JOIN 3 Tabel: Review, Pesanan, dan Pelanggan
SELECT r.id_Review, p.nama_pelanggan, pes.id_pesanan, r.rating,
r.komentar
FROM Review r
JOIN Pesanan pes ON r.id_pesanan = pes.id_pesanan
JOIN Pelanggan p ON pes.id_pelanggan = p.id_pelanggan;
```

```
-- [KELOMPOK C: 2 QUERY SUBQUERY / CTE]
```

```
-- 8. Subquery (Derived Table): Menampilkan data pelanggan dengan nilai
belanja > Rp 50.000
```

```
SELECT p.nama_pelanggan, p.no_telp, trx_besar.total_bayar
FROM Pelanggan p
JOIN (
    SELECT id_pelanggan, total_bayar
    FROM Pesanan
    WHERE total_bayar > 50000
) AS trx_besar ON p.id_pelanggan = trx_besar.id_pelanggan;
```

```
-- 9. CTE (Common Table Expression): Memfilter menu makanan yang memiliki
rating ulasan rata-rata > 3.0
```

```
WITH RataRataRating AS (
    SELECT dp.id_menu, AVG(r.rating) AS avg_rating
    FROM Review r
    JOIN Pesanan pes ON r.id_pesanan = pes.id_pesanan
```

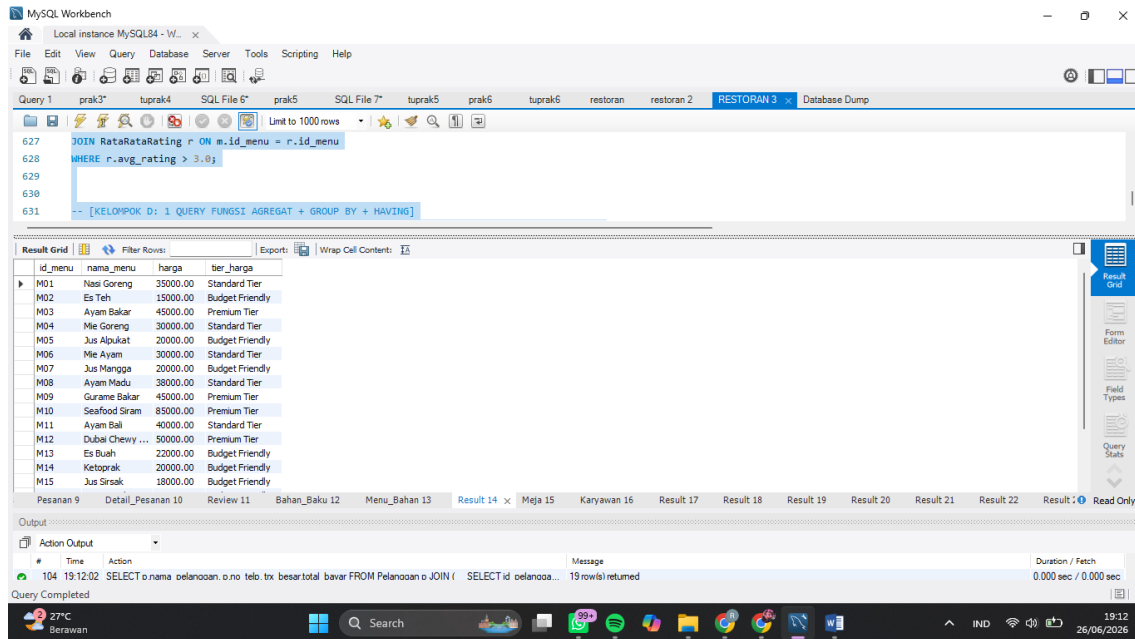
```

        JOIN Detail_Pesanan dp ON pes.id_pesanan = dp.id_pesanan
        GROUP BY dp.id_menu
    )
    SELECT m.nama_menu, r.avg_rating
    FROM Menu m
    JOIN RataRataRating r ON m.id_menu = r.id_menu
    WHERE r.avg_rating > 3.0;

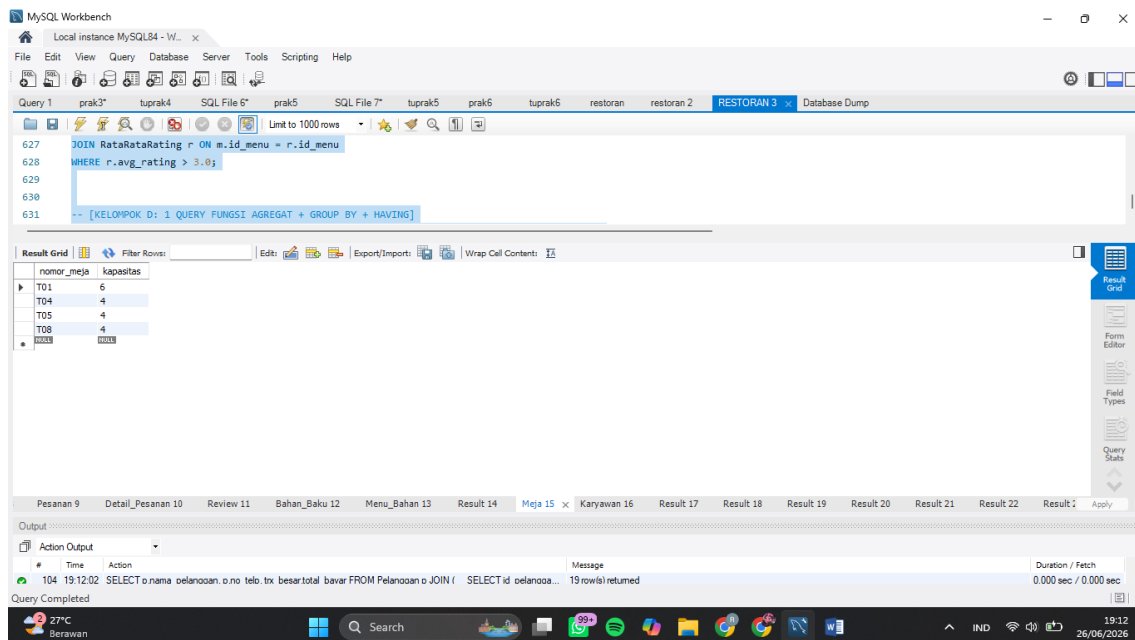
-- [KELOMPOK D: 1 QUERY FUNGSI AGREGAT + GROUP BY + HAVING]
-- 10. GROUP BY & HAVING: Mencari menu yang total porsi terjual secara
akumulatif > 3 porsi
SELECT      m.id_menu,      m.nama_menu,      SUM(dp.jumlah_porsi)      AS
total_porsi_terjual
FROM Detail_Pesanan dp
JOIN Menu m ON dp.id_menu = m.id_menu
GROUP BY m.id_menu, m.nama_menu
HAVING SUM(dp.jumlah_porsi) > 3;

```

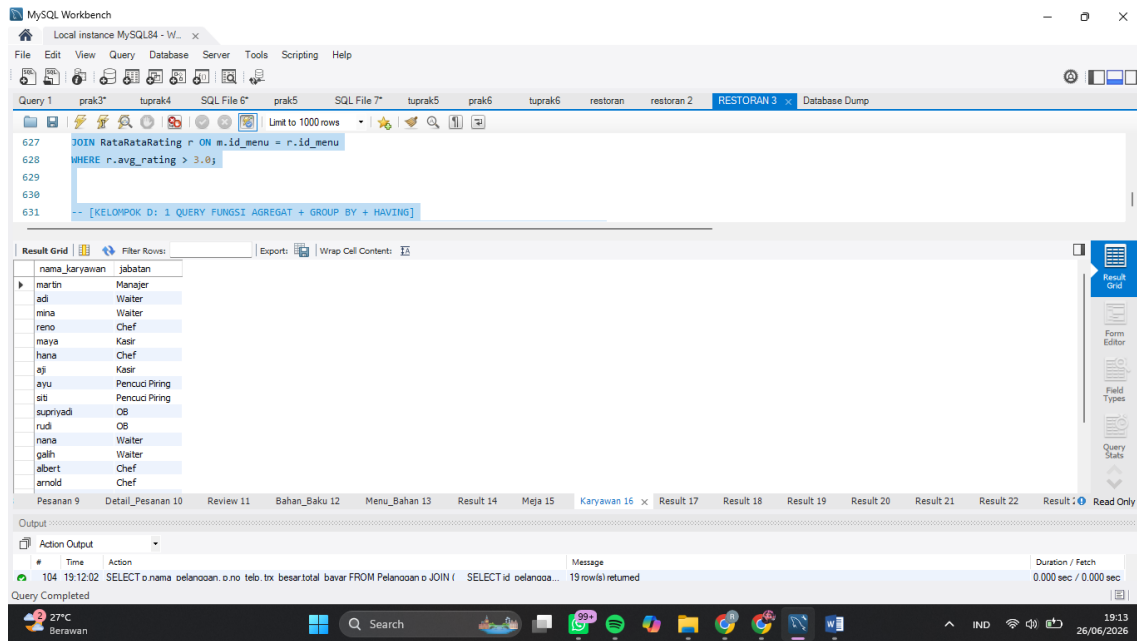
Lampiran 2. Output Query



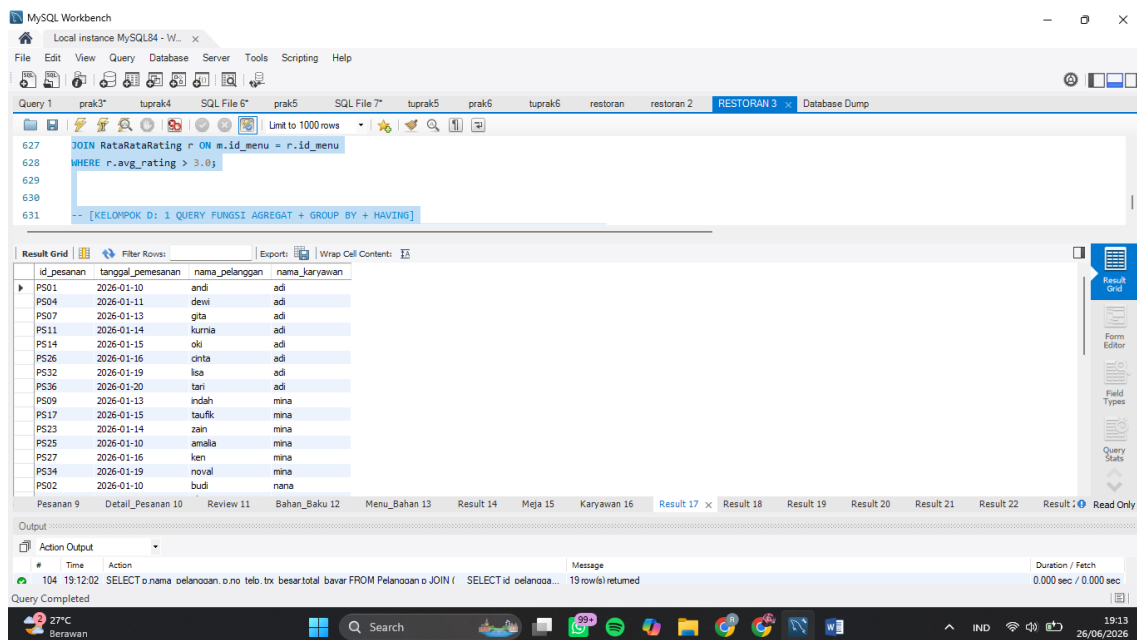
Gambar 1 Menampilkan Menu Beserta Kelas Tiering Harganya Menggunakan FUNCTION



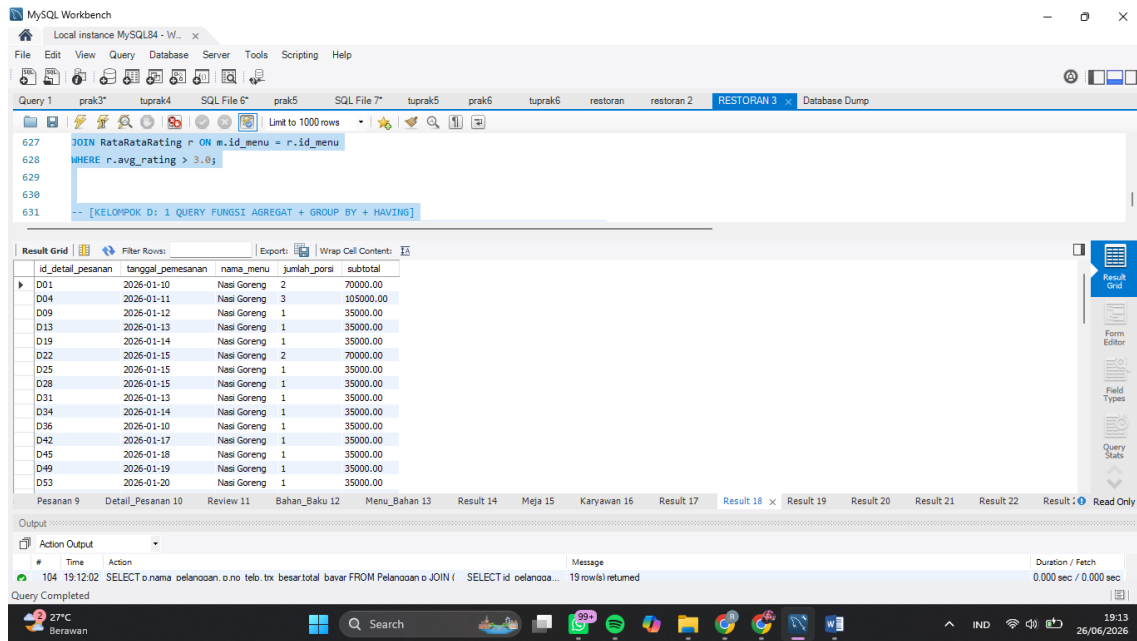
Gambar 2 Menampilkan Daftar Meja yang Memiliki Kapasitas Lebih dari 3 Orang



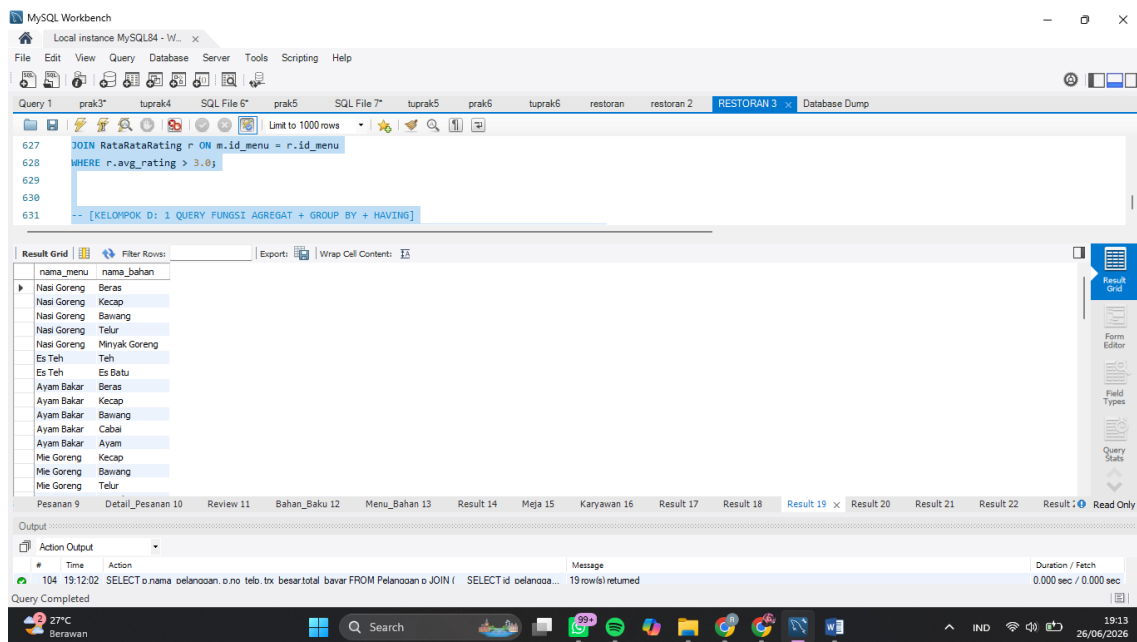
Gambar 3 Menampilkan Data Karyawan & Jabatan



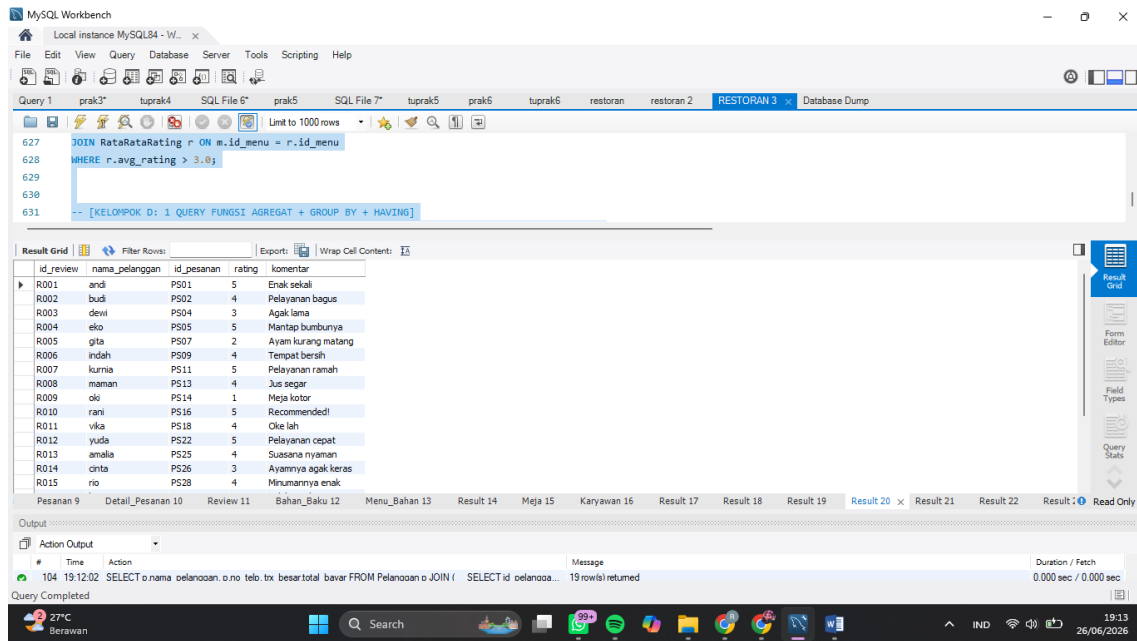
Gambar 4 JOIN 3 Tabel: Pesanan, Pelanggan, dan Karyawan



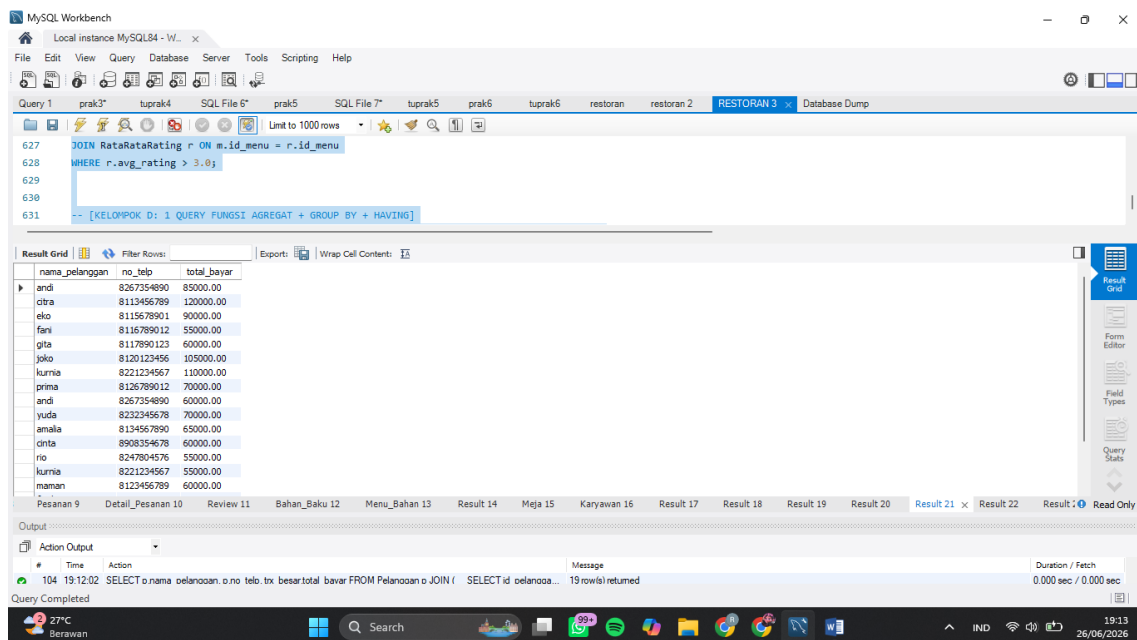
Gambar 5 JOIN 3 Tabel: Detail Pesanan, Pesanan, dan Menu



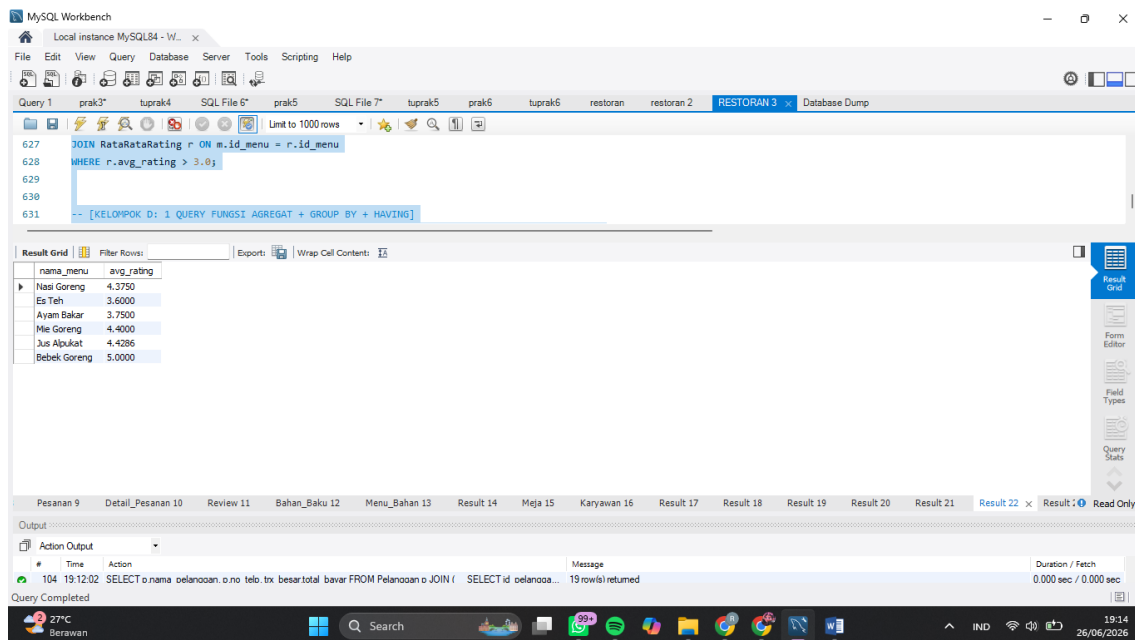
Gambar 6 JOIN 3 Tabel: Menu, Menu_Bahan, dan Bahan_Baku



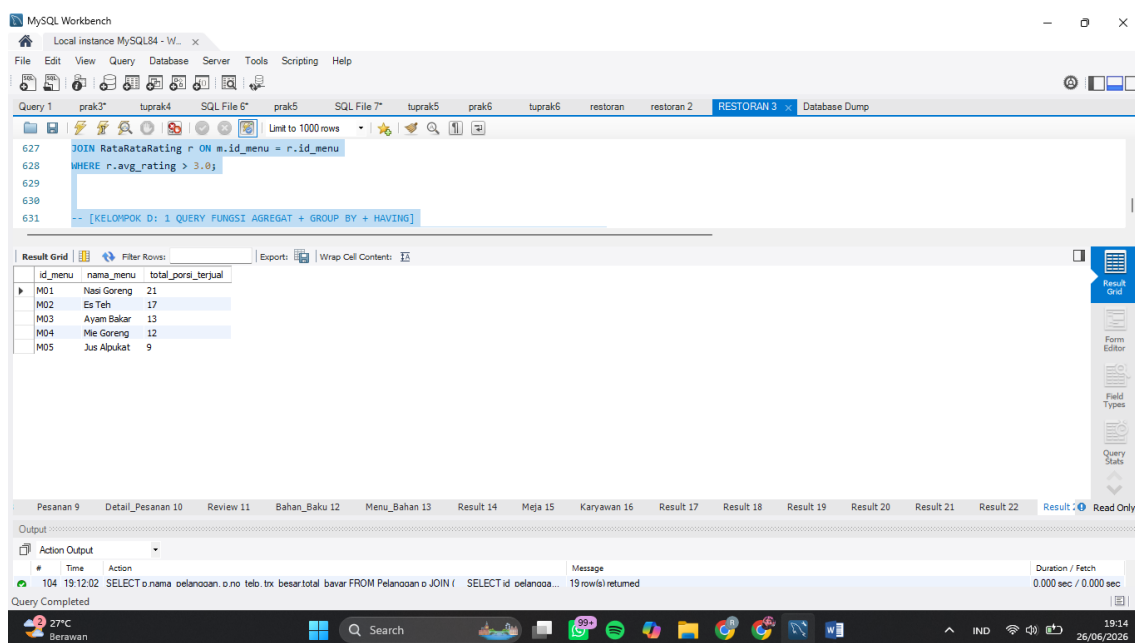
Gambar 7 JOIN 3 Tabel: Review, Pesanan, dan Pelanggan



Gambar 8 Subquery: Menampilkan Data Pelanggan dengan Nilai Belanja > Rp 50.000



Gambar 9 CTE: Memfilter Menu Makanan yang Memiliki Rating Ulasan Rata-Rata > 3.0



Gambar 10 GROUP BY & HAVING: Mencari Menu yang Total Porsi Terjual Secara Akumulatif > 3 Porsi

Lampiran 3. *Link* GitHub

<https://github.com/YonnnKun/Basis-Data-Kelompok-F>

Lampiran 4. *Link History Artificial Intelligence*

https://gemini.google.com/share/d/14gn2LlpJUKRWv5MdDxRaxUhQ0aoq_RfA?usp=sharing

https://gemini.google.com/share/d/1ZuQCDSrfI-VBR_oH8RZv2fRbn1XRyI8t?usp=sharing

<https://claude.ai/share/4e1196bb-9eee-4648-91c9-56855abc808b>